工位导航与物体分拣机器人

摘要

该作品面向实验室、办公区域中的物品整理与器材归位需求，本作品设计一套基于 ROS 与 AMD KV260 平台的双机械臂工位导航与物体分拣机器人。选题立足于重复整理工作占用人力、跨工位搬运与精细操作难以衔接等实际问题，旨在贯通机器人“到达目标区域、识别待处理物品、执行操作、确认结果”的作业流程，并探索 FPGA 在机器人视觉感知与数据通信中的应用价值。

已有研究为本作品提供了技术基础：ROS 支持机器人通信与导航集成，MoveIt 提供机械臂运动规划与操作支持；LeRobot 提供了数据采集、策略训练与部署工具。同时，KV260 官方视觉方案已展示 DPU 神经网络推理与 FPGA 图像预处理相结合的技术路径。本作品在这些基础上，重点开展面向工位整理任务的软硬件协同设计，而非将导航、双臂或推理加速单项技术作为原创成果。

系统以 ROS 为软件框架，统一组织多传感器通信、环境建图、自主定位、路径规划、移动避障与任务调度。用户通过远程终端选择目标工位并下发任务，机器人自主导航至指定区域，识别物品类别、位置及摆放状态，结合双机械臂与 LeRobot 操作流程完成抓取、搬运、分类和归位。操作结束后再次检测目标区域，对识别偏差或执行失败进行调整与重试，形成“任务下发—自主导航—加速感知—目标定位—物品操作—结果确认”的闭环。

作品的创新点主要体现在三个方面：一是面向视觉处理负载，将 DPU 目标检测推理与 FPGA 部分图像预处理、数据流处理相结合，以降低感知链路延迟为优化目标；二是将移动导航、双机械臂操作和 LeRobot 相衔接，围绕跨工位整理需求组织连续作业，并引入操作后复检机制；三是利用 FPGA 实现 OFDM 图像与文本传输，为远程图像查看和信息交互提供通信支撑，使可编程逻辑同时参与感知计算与数据传输。

本作品突出面向实际任务的系统集成与软硬件协同，为实验器材整理、仓储分拣及特殊环境辅助作业提供可扩展的技术方案。



图摘要 整车图片

第一部分　作品概述

1.1 功能与特性

本作品是一套基于 ROS 与 AMD KV260 平台的双机械臂工位导航与物体分拣机器人，面向实验室、办公区域等场所的物品整理、器材搬运和工位归位需求。系统以 ROS 为软件框架，集成移动底盘控制、激光雷达建图、机器人定位、路径规划、移动避障、深度视觉感知、任务调度及机械臂控制等功能。用户可通过键盘或远程终端控制机器人运动、选择目标工位并下发任务，机器人根据已建立的地图完成自主导航，到达指定区域后对目标物品进行识别和分拣。

系统利用深度相机获取物体的彩色图像与深度信息，并在 AMD KV260 平台的 DPU 上部署 YOLOv4 目标检测网络，实现物体类别识别。针对视觉处理过程中计算量大、处理延迟较高以及 PS 端内存占用较多的问题，利用 HLS 设计硬件加速算子，对 YOLOv4 前端和后端的部分处理过程进行加速，充分发挥 FPGA 的并行计算能力，提高视觉感知效率。系统还结合目标检测结果和深度信息，获取物体的空间位置，为机械臂抓取和分拣提供依据。

在机械臂控制方面，一只机械臂结合 LeRobot 完成物品抓取与分拣，并将部分网络部署至 DPU 进行加速，Transformer 相关计算由 KV260 的 PS 端完成，实现软硬件协同处理；另一只机械臂由 Spartan-7 控制，并通过 KV260 创建 ROS 节点，实现键盘控制、MoveIt 运动规划等功能。与此同时，系统基于 Zynq-7020 与 AD9361 搭建 LWIP+OFDM 通信系统，通过网口和串口与 KV260 连接，实现文本信息、任务状态及 YOLO 识别图像的传输，为远程查看和任务交互提供支持。

任务执行完成后，系统再次检测目标区域，对识别错误、抓取失败或归位偏差进行调整和重试，形成“任务下发—自主导航—视觉识别—目标定位—机械臂操作—结果确认”的闭环作业流程。作品具有模块化、软硬件协同和可扩展等特点，可拓展应用于实验器材整理、仓储分拣及特殊环境辅助作业等场景。

1.2 应用领域

本作品主要应用于实验室、办公区域、教学实训场所及小型仓储空间中的物品整理、器材搬运与分类归位。针对实验室中试剂瓶、实验器材、工具等物品种

类多、摆放位置分散、重复整理工作量大的问题，机器人能够根据任务指令自主导航至指定工位，完成物品识别、抓取、搬运、分类和归位，并通过操作后复检提高任务执行的准确性。在办公区域中，可用于文件盒、办公用品及常用工具的收集与整理，减少人员在不同工位之间往返搬运的工作量。

依托 ROS 的导航与任务调度能力、双机械臂的协同操作能力以及 AMD KV260 平台的视觉计算加速能力，本作品还可拓展应用于小型仓储分拣、生产线物料转运、智能实验室管理和高校机器人教学实践等场景。在高温、粉尘、有害物质或人员不宜频繁进入的特殊环境中，机器人可通过远程终端接收任务并完成一定范围内的物品识别与搬运作业，从而降低人员劳动强度和作业风险。通过更换末端执行器、扩展目标识别类别和调整导航地图，系统能够适应不同物品形态、工位布局及任务流程，具有较好的场景适应性和应用拓展价值。

1.3 主要性能指标

1.3.1 各工程性能对比与验收汇总

表 1-1 各工程性能对比与验收汇总

工程/阶段	指标	优化前/对照配置	优化后/结果	改善与验收说明
OFDM：TCP 图像传输	平均传输延迟	426.21 ms	159.31 ms	下降 62.6%；均为 Nagle 配置，属于版本演进对比
OFDM：TCP 图像传输	应用有效吞吐	173.36 kB/s	463.45 kB/s	提升至 2.67 倍；不能全部归因于单项修改
OFDM：交织版板测	连续图像传输	—	400/400 成功；586.72 kB/s	每张 73,944 字节；记录 5 次重试、5 次 RX 丢弃
OFDM：视频回环	分辨率、帧率与完整性	—	640×480，约 7 FPS；840/840 帧完整	120 秒合成 JPEG 负载；2 帧迟到，严格实时验收未通过
OFDM：ACK 等待对照	总耗时 / 成功次数	100 ms 等待：75.01 s；400/400	1 s 等待：505.32 s；399/400	负向实验：延长等待未改善本轮结果

HLS：图像预处理	CDF 子模块最大周期数	12,549 周期	3,333 周期	下降 73.4%；不代表整帧加速比
HLS：图像预处理	整 IP OOC 时序余量 WNS	0.452 ns	0.231 ns	仍通过，但余量下降；局部优化不等于全 IP 时序改善
HLS：YOLO 后处理	整 IP OOC 时序余量 WNS	0.798 ns	1.535 ns	增加 0.737 ns；代价是 suppress 最大延迟由 3 增至 23 周期
HLS：基础平台布线	全局 WNS	0.038 ns	0.157 ns	增加 0.119 ns；不含最终 DPU 链接
DPU：最终链接实现	全局 WNS / WHS	—	0.001 / 0.007 ns	TNS、THS 均为 0；通过现有约束，但时序余量很薄
视觉：帧槽优化	端到端延迟 P50 / P95	1 runner、3 槽：505 / 538 ms	1 runner、2 槽：349 / 381 ms	P50 下降约 30.9%；吞吐基本保持
视觉：runner 对照	短测吞吐	1 runner、3 槽：6.23 FPS	2 runner、3 槽：6.50 FPS	提升约 4.2%，增益有限
视觉：真实相机板测	连续运行吞吐与帧数	—	90.11 秒，568 帧，6.30 FPS	P50/P95 约 349/380 ms，取最近帧窗口
视觉：阶段耗时	前处理 / DPU / 后处理	—	6.31 / 157.35 / 34.08 ms	最近 256 帧均值；包含同步、调度及搬运开销
导航：错误初始化	180 °反向朝向门控	错误放行	成功拒绝	加入自由射线矛盾检查；不保证解决对称环境定位多解
导航：集成定位精度	到点位置 / 朝向误	无同口径基线	首次：0.017 m / 0.100 rad；另一目	隔离运动学集成结果，不是实车精度

	差		标：0.074 m / 0.086 rad	
导航：任务恢复	单次初始化后的自动恢复	曾出现异常后锁定、重复初始化问题	单个公开目标自动恢复成功，暂停零速度检查通过	属状态机与集成验收；仍有后续实车卡住反馈
机械臂：ACT 视觉计算	单次视觉特征提取延迟	PS 执行：240 ms	DPU 执行：60 ms	下降 75.0%；仅为视觉子模块，不代表整套 ACT 加速比
机械臂：ACT 协同推理	单次动作块生成延迟	全 PS 执行：480 ms	DPU 视觉骨干＋PS Transformer：300 ms	下降 37.5%，速度提升至 1.60 倍；计时应包含预处理、数据交接及动作后处理
机械臂：ROS—Spartan-7 接口	指令发送至接收确认的平均延迟	逐条发送、同步等待：20 ms	优化报文组织与收发调度：12 ms	下降 40.0%；接收确认不等于机械臂到位，协议及计时边界待验证
机械臂：抓取分拣任务	单次抓取—搬运—放置平均耗时	对照流程：12 s	调整调度与等待策略：10 s	下降 16.7%；需保持对象、路径、速度限制和成功判据一致
机械臂：任务闭环	固定测试集任务成功率	16/20 次，80%	18/20 次，90%	提高 10 个百分点；不是计算加速的必然收益，需分别统计首次成功与重试后成功

1.3.2 主要资源指标

工程/对比范围	资源指标	优化前	优化后	变化
OFDM：RS 编解码模块合计	LUT / FF	1,415 / 1,143	1,798 / 1,204	增加 383 LUT、61 FF；换取交织能力，BRAM 不变

HLS：CDF 子模块	LUT / FF / DSP	1,110 / 889 / 3	708 / 261 / 0	LUT 下降 36.2%
HLS：首轮 YOLO 后处理整 IP	LUT / FF / DSP	9,260 / 6,068 / 8	7,834 / 6,436 / 8	LUT 下降 15.4%，FF 增加
HLS：基础平台布线	LUT / FF / DSP	27,306 / 32,488 / 63	23,104 / 31,106 / 61	分别下降 15.39% / 4.25% / 3.17%
视觉：3 槽→2 槽	应用 CMA 缓冲分配	3 槽	2 槽	减少 1,424,176 字节，约 1.36 MiB；不等于进程 RSS 下降
视觉：90 秒板测	最终进程 RSS	—	约 282.5 MiB	单次运行观测值，不是严格峰值上限
导航	CPU / 内存/ 执行耗时	—	缺少可比实测数据	不能声称资源节省

表 1-2 主要资源指标

1.4 主要创新点

1.4.1 面向机器人视觉感知的软硬件协同加速

针对 ROS 视觉感知任务中图像处理量大、目标检测推理延迟较高的问题，利用 AMD KV260 平台的异构计算架构，在 DPU 上部署 YOLOv4 目标检测网络，并利用 FPGA 的并行处理能力完成部分图像预处理、数据流处理及检测结果处理。同时，结合 HLS 设计硬件加速算子，减少视觉数据在处理器端的重复计算和内存占用，实现算法、软件与硬件资源的协同优化，提高视觉感知链路的运

行效率。

1.4.2 面向跨工位整理任务的移动操作一体化设计

将 ROS 自主导航、SLAM 建图、定位避障、任务调度和双机械臂操作流程进行统一集成，使机器人能够完成从目标工位选择、自主移动、物品识别，到目标定位、抓取、搬运、分类和归位的连续作业。该设计面向实际跨工位整理需求，增强了移动平台与操作机构之间的协同性，避免导航系统与机械臂操作流程相互割裂。

1.4.3 双机械臂异构控制与协同操作设计

针对两套机械臂的控制需求，采用不同硬件平台和软件方式进行协同设计。一只机械臂结合 LeRobot 完成物体抓取与分拣，并将部分网络部署至 KV260 的 DPU 进行加速，Transformer 相关计算由 KV260 的 PS 端完成，实现软硬件协同推理；另一只机械臂由 Spartan-7 进行控制，并在 KV260 上创建 ROS 节点，实现键盘控制、MoveIt 运动规划及相关操作功能。通过双机械臂的功能分工，提升系统对不同操作任务的适应能力。

1.4.4 引入操作结果复检机制，提升任务可靠性

在完成物品抓取、搬运或归位后，系统对目标区域进行再次视觉检测，根据物品类别、位置和摆放状态判断任务是否完成。当出现识别偏差、抓取失败或放置位置不准确等情况时，系统能够根据检测结果调整操作策略并进行重试，形成“任务下发—自主导航—视觉识别—目标定位—物品操作—结果确认”的闭环作业流程，提高系统执行的准确性和可靠性。

1.4.5 融合 FPGA 通信能力，拓展远程交互功能

基于 Zynq-7020 与 AD9361 搭建 LWIP+OFDM 通信系统，并通过网口和串口与 KV260 进行连接，实现任务文本、系统状态及 YOLO 识别图像等信息的传输。该设计将 FPGA 的通信处理能力应用于机器人远程交互场景，为远程图像查看、任务下发、状态反馈和信息交互提供支持，拓展了机器人在复杂或不便直接进入环境中的应用能力。

1.4.6 采用模块化架构，支持多场景扩展

系统采用 ROS 模块化架构，将移动导航、视觉感知、目标检测、通信传输、机械臂控制和任务调度等功能进行独立封装。根据应用需求，可以扩展传感器、识别类别、末端执行器、导航地图和任务流程。通过更换机械臂末端工具、调整识别模型及配置不同工位任务，系统能够拓展至实验器材整理、办公用品归位、小型仓储分拣及特殊环境辅助作业等场景。

1.5 设计流程

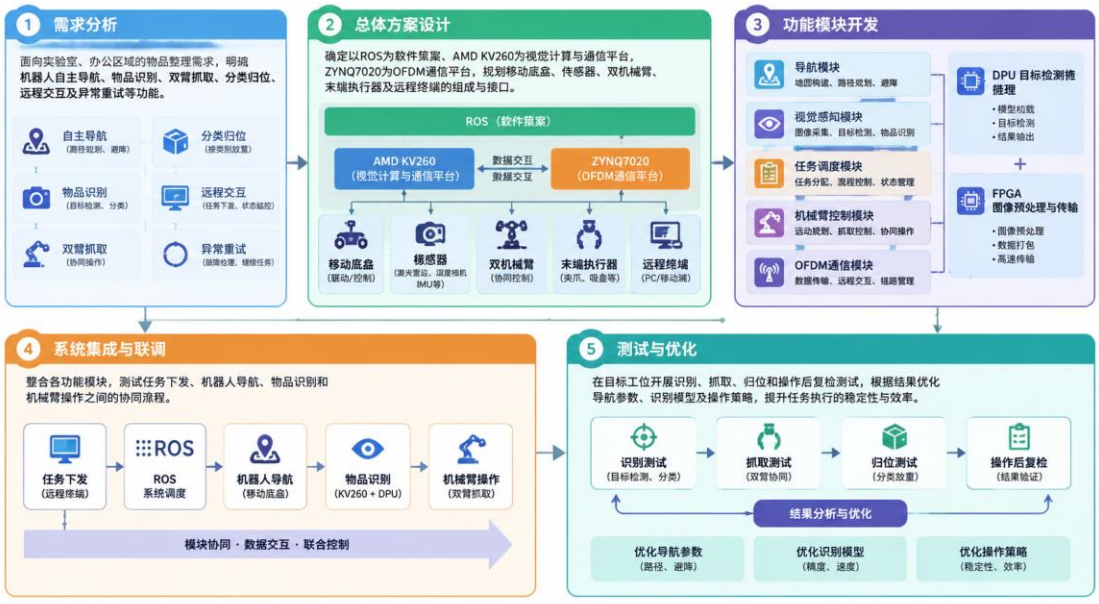


图 1-1 作品设计流程

1.5.1 需求分析

面向实验室、办公区域的物品整理需求，明确机器人自主导航、物品识别、双臂抓取、分类归位、远程交互及异常重试等功能。

1.5.2 总体方案设计

确定以 ROS 为软件框架、AMD KV260 为视觉计算与通信平台，ZYNQ7020 为 OFDM 通信平台，规划移动底盘、传感器、双机械臂、末端执行器及远程终端的组成与接口。

1.5.3 功能模块开发

分别设计导航、视觉感知、任务调度、机械臂控制和 OFDM 通信模块；利

用 DPU 进行目标检测推理，并由 FPGA 完成部分图像预处理及数据传输。

1.5.4 系统集成与联调

整合各功能模块，测试任务下发、机器人导航、物品识别和机械臂操作之间的协同流程。

1.5.5 测试与优化

在目标工位开展识别、抓取、归位和操作后复检测试，根据结果优化导航参数、识别模型及操作策略，提升任务执行的稳定性与效率。

第二部分　系统组成及功能说明

2.1 整体介绍

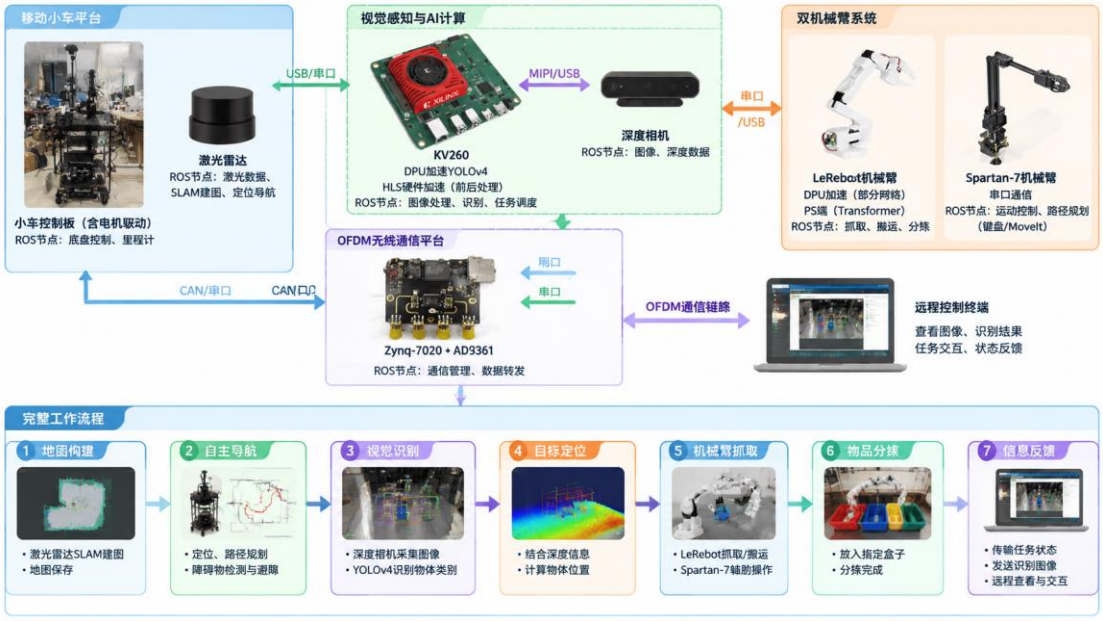


图 2-1 系统总体架构框图

2.1.1 移动导航平台设计

系统以 AMD KV260 开发平台为主要计算平台，部署 Ubuntu 操作系统及 ROS 软件环境。移动小车控制板负责底盘电机、编码器等执行部件的控制，并通过 ROS 节点向上层系统发布小车的运动状态，同时接收速度控制指令。激光雷达安装在移动小车上，用于采集周围环境的距离信息，并通过相应的 ROS 节点将扫描数据传输至导航系统。

在建图阶段，机器人通过键盘控制小车在工作区域内移动，ROS 系统根据激

光雷达采集的环境信息进行 SLAM 建图，建立实验室或办公区域的二维环境地图。建图完成后，将地图数据保存至系统中。在导航阶段，机器人根据保存的地图进行定位，并结合目标工位位置完成路径规划。导航系统向底盘控制节点发送速度指令，使小车按照规划路径移动；当检测到障碍物时，系统重新规划局部路径，实现移动过程中的避障控制。

2.1.2 深度视觉感知设计

系统采用深度相机获取目标区域的彩色图像和深度数据，并在 ROS 中创建对应的相机驱动节点，将图像信息和深度信息发布给视觉处理模块。彩色图像首先经过格式转换、尺寸调整等预处理，然后输入 YOLOv4 目标检测网络，对工位上的物体进行识别，获得物体类别和二维图像位置。

为了提高 YOLOv4 的运行效率，系统将目标检测网络部署到 KV260 平台的 DPU 中执行。针对 YOLOv4 前端图像处理和后端结果处理过程中存在的计算量较大、PS 端内存占用较高等问题，利用 HLS 设计相应的硬件加速算子，将部分图像预处理、数据流处理及检测结果后处理任务放置在 FPGA 可编程逻辑部分执行。这样可以减少数据在 PS 端和 PL 端之间的重复搬运，降低处理器负担，并提高视觉处理链路的并行度。

在获得 YOLOv4 检测框后，系统根据检测框中心点或指定区域读取深度相机对应的深度值，结合相机内参计算目标物体与相机之间的空间位置。最终输出包括物体类别、图像坐标、深度信息及目标空间位置等内容，为后续机械臂运动规划和抓取操作提供依据。

2.1.3 图像与文本通信系统设计

系统采用 Zynq-7020 与 AD9361 搭建无线通信平台，在 Zynq-7020 上部署 LWIP 协议栈和 OFDM 通信系统。Zynq-7020 通过网口或串口与 KV260 进行连接，接收来自 ROS 系统的任务状态、控制指令和识别结果等文本信息，同时获取深度相机采集的 YOLO 识别图像。

在发送过程中，系统首先对图像数据和文本数据进行封装，再通过 LWIP 完成网络数据传输，随后交由 OFDM 通信模块进行调制和无线发送。接收端完成信号解调、数据恢复和信息解析，将接收到的图像和文本内容传输至远程终端。通过该通信链路，用户可以查看机器人当前识别到的物体图像，获取任务执行状

态，并实现一定程度的远程控制与信息交互。

2.1.4 双机械臂操作系统设计

系统配置两套机械臂，分别承担不同的操作和控制任务。其中，一只机械臂结合 LeRobot 完成物体分拣操作。系统通过摄像头采集机械臂工作区域的图像，并利用 LeRobot 相关流程完成数据采集、操作策略处理和机械臂动作执行。针对网络推理计算量较大的问题，将 LeRobot 的部分网络部署到 KV260 的 DPU 上加速执行，Transformer 相关计算则由 KV260 的 PS 端完成，从而构成 PS 端与 DPU 协同工作的软硬件加速架构。

该机械臂根据视觉模块输出的物体类别和空间位置，确定抓取目标及抓取顺序，完成靠近目标、调整姿态、抓取物体和放置到指定盒子等动作。物品放置完成后，系统可通过视觉模块对目标区域进行再次检测，判断物体是否已经正确分拣。

另一只机械臂由 Spartan-7 平台进行控制，并在 KV260 上创建对应的 ROS 控制节点，实现机械臂控制系统与 ROS 框架之间的数据交互。该机械臂可以通过键盘发送控制指令，也可以结合 MoveIt 完成机械臂的运动规划、目标位姿设定和轨迹控制，用于辅助操作、实验验证或后续功能扩展。

2.1.5 ROS 节点与任务调度设计

系统采用模块化 ROS 节点结构，将移动底盘、激光雷达、深度相机、目标检测、机械臂控制和通信功能分别封装为独立节点。各节点通过 ROS 话题、服务或动作接口进行数据交换。例如，激光雷达节点发布环境扫描数据，导航节点根据地图和定位信息生成路径，底盘节点接收速度指令并控制小车运动；深度相机节点发布图像和深度数据，YOLO 节点输出物体检测结果，机械臂节点根据目标位置执行抓取操作。

任务调度模块负责协调各节点的运行顺序。当用户下发分拣任务后，系统首先控制小车前往目标工位，到达指定位置后启动视觉识别流程。识别完成后，任务调度模块将物体类别和位置发送给相应机械臂，机械臂执行抓取与归位操作。完成操作后，视觉系统重新检测目标区域，并将检测结果反馈给任务调度模块。如果发现物体未被抓取、放置位置不准确或目标区域仍存在待处理物体，系统则重新规划操作步骤并执行调整。

2.1.6 系统整体工作流程

系统的完整工作过程为：首先通过键盘或远程终端下发目标工位和分拣任务；随后，移动小车根据已保存地图完成定位、路径规划和自主导航；到达目标区域后，深度相机采集环境图像和深度信息，YOLOv4 识别目标物体并获取其空间位置；任务调度模块根据识别结果选择对应的机械臂执行抓取和分拣操作；在操作过程中，KV260 负责视觉推理、图像处理及部分 LeRobot 网络加速，Zynq-7020+AD9361 负责图像和文本信息的 OFDM 通信；最后，系统对分拣结果进行复检，并将任务状态和识别图像反馈至远程终端，形成完整的闭环作业流程。

2.2 系统组成

本作品由移动小车平台、激光雷达、深度相机、AMD KV260 视觉计算平台、双机械臂系统、Zynq-7020+AD9361 无线通信平台、远程控制终端及相关传感器组成。系统以 ROS 为整体软件框架，负责移动底盘、激光雷达、深度相机和机械臂等设备之间的通信、数据处理与任务调度，实现机器人从自主导航、物体识别到双臂分拣的完整作业流程。

移动小车控制板和激光雷达上部署 ROS 相关节点，通过键盘控制可以实现小车的前进、后退、转向及停止等基本运动。在此基础上，系统利用激光雷达完成环境感知和 SLAM 建图，并支持地图保存。完成地图构建后，机器人可以根据目标工位进行定位、路径规划和自主导航，在运动过程中实现障碍物检测与避障控制，为后续物品分拣任务提供移动平台支持。

深度相机与 KV260 平台共同构成系统的视觉感知模块。系统为深度相机创建 ROS 节点，获取场景图像及物体深度信息，并在 KV260 的 DPU 上部署 YOLOv4 目标检测网络，实现对工位物品的类别识别。同时，利用 HLS 设计硬件加速算子，对 YOLOv4 的前端和后端处理过程进行加速，减少部分数据处理对 PS 端的依赖和内存占用，从而提高目标检测效率。结合深度相机输出的深度数据，系统能够获得物体的类别、空间位置及相关摆放信息，为机械臂抓取和分拣提供依据。

系统采用 Zynq-7020 与 AD9361 搭建 LWIP+OFDM 通信平台，并通过网口和串口与 KV260 进行连接，实现机器人内部不同平台之间的数据交互。该通信系统能够传输任务状态、控制指令等文本信息，同时将深度相机采集并经过

YOLO 识别的图像传输至远程终端，便于用户查看现场情况、获取识别结果和进行远程任务交互。

在物体操作部分，系统配置两套机械臂。其中一只机械臂部署 LeRobot 操作流程，并利用 KV260 对 LeRobot 的部分网络进行 DPU 加速，Transformer 相关部分由 KV260 的 PS 端完成，实现软硬件协同计算。该机械臂主要根据视觉识别结果完成工位物体的抓取、搬运和分拣，将目标物品放入指定盒子中。另一只机械臂由 Spartan-7 开发板进行控制，同时在 KV260 上创建对应的 ROS 节点，串口通信使其能够通过键盘、MoveIt 等方式进行运动控制和路径规划，完成辅助操作或扩展性实验。

系统运行时，用户首先通过键盘或远程终端控制小车前往指定工位。机器人根据已保存地图完成定位、路径规划和自主导航，到达目标区域后，深度相机采集现场图像，YOLOv4 识别物体类别，结合深度信息计算目标位置。随后，任务调度模块将识别结果传递给相应机械臂，机械臂完成物品抓取并将其分拣至指定盒子。操作过程中，系统还可以通过 OFDM 通信链路传输文本信息和识别图像，实现任务状态反馈与远程查看，最终形成“地图构建—自主导航—视觉识别—目标定位—机械臂抓取—物品分拣—信息反馈”的完整系统流程。

2.3 硬件系统介绍

2.3.1 硬件整体介绍

本作品硬件系统由移动机器人平台、环境与视觉传感器、AMD KV260 视觉计算平台、双机械臂、Zynq-7020+AD9361 通信平台及远程交互设备组成，各部分通过网口、串口及相应控制接口连接，共同完成移动导航、物体识别、机械臂操作和信息传输任务。

移动机器人平台由小车底盘及其控制板构成，负责执行前进、后退、转向和停止等运动指令。激光雷达安装于小车，用于采集周围环境信息，支持机器人进行环境建图、定位、路径规划和移动避障。深度相机用于采集工位物体的彩色图像与深度数据，为目标识别和空间定位提供信息；系统另设摄像头用于机械臂工作区域的图像采集。

AMD KV260 作为系统的主要视觉计算与控制平台，承担 ROS 节点运行、

视觉数据处理、YOLOv4 目标检测及部分机械臂控制等任务。平台利用 DPU 加速目标检测及 LeRobot 部分网络的推理，并通过 FPGA 可编程逻辑执行 HLS 设计的图像处理加速算子。KV260 与小车控制板、传感器、机械臂控制平台及通信平台连接，实现数据交互和任务协调。

系统配备两套机械臂，分别承担物品分拣和辅助操作任务。其中一套机械臂结合 LeRobot 操作流程，由 KV260 完成相关计算与控制；另一套机械臂由 Spartan-7 控制，并与 KV260 上的 ROS 控制节点连接，支持键盘控制和 MoveIt 运动规划。Zynq-7020 与 AD9361 组成无线通信平台，通过网口和串口与 KV260 连接，用于传输文本信息及目标识别图像。远程终端用于任务下发、图像查看和状态监控。

上述硬件共同构成“移动平台负责到达工位、传感器负责环境与目标感知、KV260 负责计算和协调、双机械臂负责物品操作、通信平台负责信息传输”的系统架构，为机器人完成工位导航与物体分拣提供硬件基础。

2.3.2 机械设计介绍

本作品的机械系统主要由 LeRobot 开源 SO-101 机械臂、微雪 RoArm-M3 机械臂及碳纤维机器人底座组成。两套机械臂安装在碳纤维板构成的机器人底座上，形成双机械臂协同操作结构。SO-101 机械臂主要用于结合 LeRobot 完成工位物品的识别、抓取、搬运及分类放置等操作；微雪 RoArm-M3 机械臂主要用于辅助作业、运动控制实验及基于 ROS 和 MoveIt 的轨迹规划。碳纤维板具有质量轻、强度高、结构稳定等特点，可为两套机械臂、控制器、摄像头及相关电子设备提供安装和支撑，并便于根据不同任务需求调整机械臂的位置与安装方式。整体机械结构采用模块化设计，能够满足双臂协同、视觉采集和物品分拣等功能需求，同时为后续更换末端执行器、扩展传感器和优化机械布局提供条件。

2.4 软件系统介绍

2.4.1 软件整体介绍

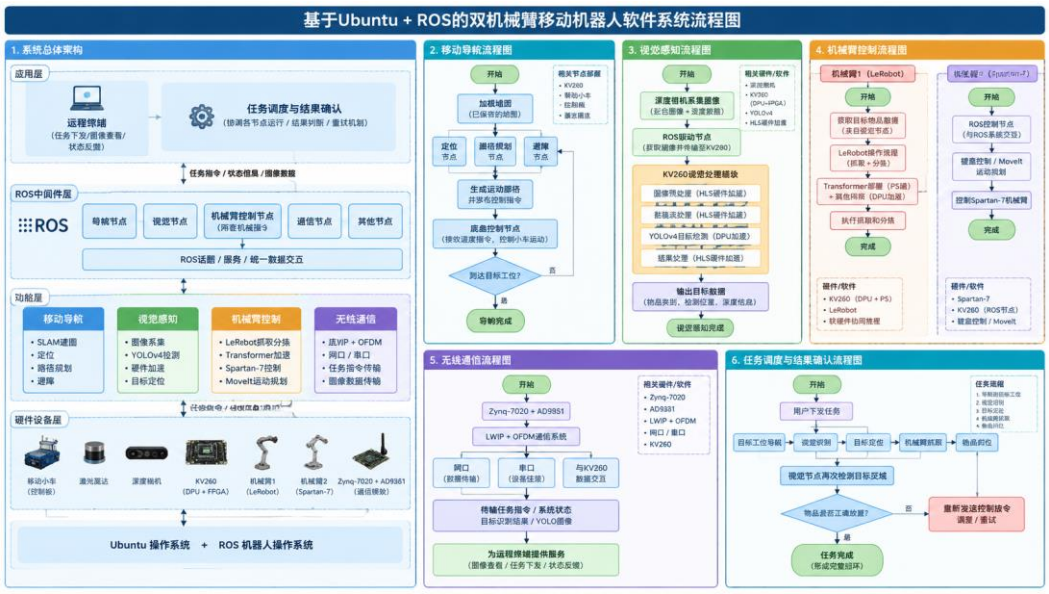


图 2-2 软件系统整体架构

本作品的软件系统基于 Ubuntu 操作系统和 ROS 机器人操作系统构建，采用模块化、分层式的软件架构，将移动导航、视觉感知、无线通信、双机械臂控制和任务调度等功能进行统一集成。系统通过 ROS 节点实现各硬件设备之间的数据通信和功能协同，使移动小车、激光雷达、深度相机、机械臂及远程终端能够按照任务流程协同工作。

在移动导航部分，系统在 KV260 平台、移动小车控制板及激光雷达上部署相应的 ROS 节点。底盘控制节点负责接收速度指令并控制小车运动，激光雷达节点负责发布环境扫描数据。系统利用 SLAM 算法完成工作区域地图构建，并将地图保存至指定位置。在导航过程中，定位、路径规划和避障等节点根据已保存地图、传感器数据和目标工位信息，生成小车的运动路径并发布控制指令，实现机器人从当前位置到目标工位的自主移动。

在视觉感知部分，系统为深度相机创建 ROS 驱动节点，获取彩色图像和深度数据，并将其传输至 KV260 视觉处理模块。YOLOv4 目标检测网络部署在 KV260 的 DPU 上，用于识别工位上的目标物品。针对图像预处理、数据流处理以及检测结果处理等环节，系统利用 HLS 设计硬件加速算子，在 FPGA 可编程

逻辑部分完成并行计算，从而减少 PS 端的计算和内存占用。视觉节点最终输出物品类别、检测位置和深度信息，为机械臂抓取提供目标数据。

在机械臂控制部分，系统分别为两套机械臂设计相应的软件控制模块。其中，一只机械臂采用 LeRobot 相关操作流程完成物品抓取和分拣，并将部分网络部署到 KV260 的 DPU 中进行加速，Transformer 部分由 KV260 的 PS 端完成，实现软硬件协同推理。另一只机械臂由 Spartan-7 控制，并在 KV260 上创建 ROS 控制节点，实现与 ROS 系统的数据交互，同时支持键盘控制和 MoveIt 运动规划等功能。

在通信部分，系统基于 Zynq-7020 和 AD9361 搭建 LWIP+OFDM 通信软件系统，并通过网口、串口与 KV260 进行数据连接。通信模块负责传输任务指令、系统状态、目标识别结果以及深度相机采集的 YOLO 识别图像，为远程终端提供图像查看、任务下发和运行状态反馈功能。

系统还设置了任务调度与结果确认模块，用于协调各 ROS 节点的运行顺序。当用户下发任务后，系统依次执行目标工位导航、视觉识别、目标定位、机械臂抓取和物品归位等操作。任务完成后，视觉节点再次检测目标区域，根据检测结果判断物品是否正确放置；若出现识别错误、抓取失败或归位偏差，任务调度模块将重新发送控制指令，组织机器人进行调整和重试，最终形成完整的软件闭环。

2.4.2 移动导航部分

我们团队依照 https://github.com/amd/Kria-RoboticsAI 的相关指引在 KV260 上安装了 Ubuntu22.04，部署了 PYNQ-DPU 以及机器人操作系统（ROS2/ROS1）。由此展开移动导航部分。

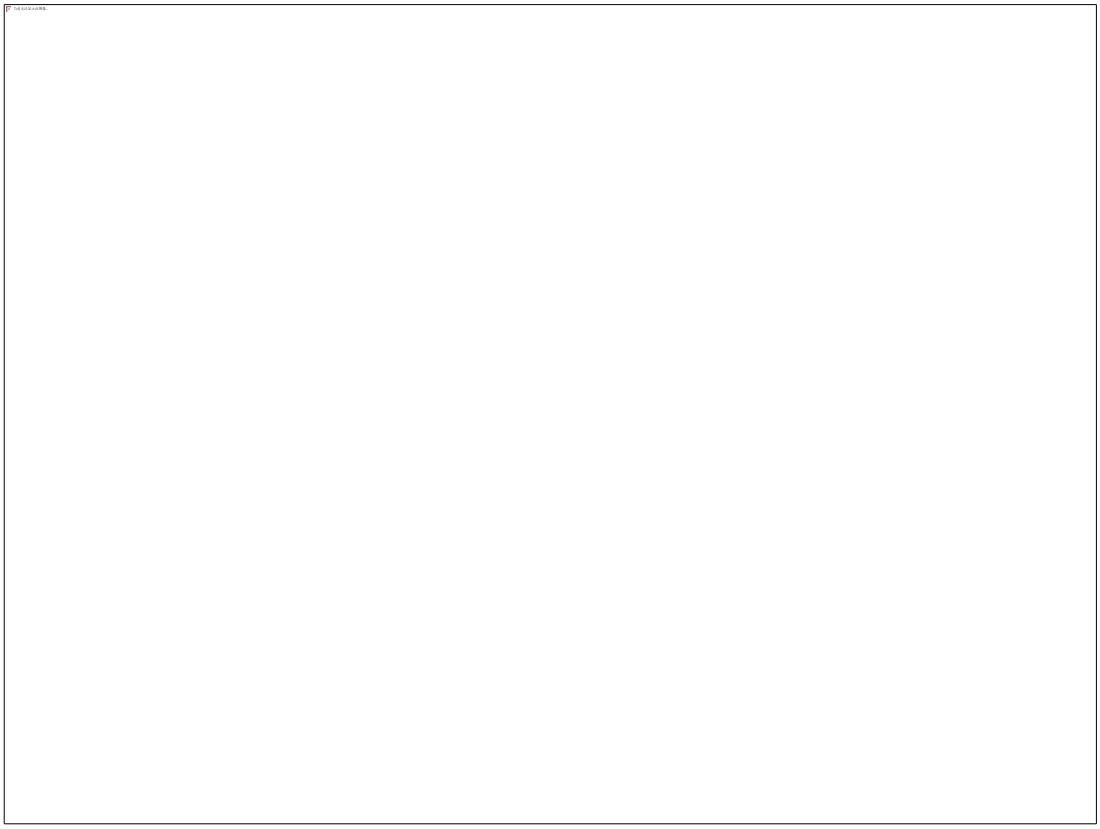


图 2-3 移动导航子系统架构

2.4.2.1 总体方案

移动导航子系统面向实验室、办公区域等室内工位场景，部署于 AMD KV260 的 ARM64 Docker 环境中，采用 ROS 1 Melodic 导航架构，以二维激光雷达、底盘运动反馈和 IMU 为主要输入，完成底盘控制、运动状态估计、激光建图、地图定位、路径规划和速度执行。系统不以视觉识别为核心，而是为机器人到达目标工位提供可重复的移动平台能力。

从职责上分为六层：硬件执行层（X3plus 全向底盘、控制板、串口通信）、驱动层（Rosmaster 库、底盘 ROS 驱动、YDLidar 驱动）、状态估计层（原始里程计、Madgwick 滤波、EKF）、建图与定位层（Gmapping、map_server、AMCL）、路径规划层（move_base、全局规划器、DWA、代价地图）以及管理与门控层（navigation_guard、启动脚本、RViz、起点书签）。主要数据流为：底盘控制板输出运动速度反馈与 IMU 数据，经原始里程计和 EKF 融合形成 /odom 与运动 TF；激光雷达发布 /scan，建图时送入 Gmapping，导航时结合已保存地图送入 AMCL；用户目标经 navigation_guard 转发至 move_base，由全局规划和

DWA 生成内部速度指令，再经守卫检查后发布至 /cmd_vel，最终由底盘驱动和控制板执行。

2.4.2.2 底盘控制与运动状态估计

底盘采用麦克纳姆/全向运动模型，上层发送机器人本体速度而非逐个电机转速：Twist.linear.x 表示前后速度，Twist.linear.y 表示横向速度，Twist.angular.z 表示绕竖直轴角速度。驱动订阅 /cmd_vel，将三个分量传入 Rosmaster 的 set_car_motion()，经串口发送至底盘控制板，串口波特率为 115200。底盘反馈包括车体运动速度、IMU 加速度与角速度、磁力计、电池电压、控制板版本及机械臂关节状态，发布循环设计节奏约 20 Hz。

原始里程计不是直接把控制指令当作里程计，而是读取控制板报告的运动速度后进行平面积分，并对 X、Y 方向设置比例修正（当前配置分别为 1.10 和 0.95）。随后系统进行两级处理：Madgwick IMU 滤波处理惯性数据，当前配置不使用磁力计参与姿态估计；EKF 融合底盘速度和 IMU 信息，输出 /odom，配置频率 20 Hz，启用二维模式。驱动还包含软件速度看门狗：有效速度指令到来时更新时间，超过约 0.5 s 无新指令时发送零速度，看门狗线程约每 0.1 s 检查一次；收到 NaN、正负无穷等异常值时发送零速度，正常关闭节点时也发送零速度。该机制用于失联防护和异常值保护，软件发出零速度不等同于测得物理停车，硬件急停和实测停车时间/距离仍需单独验证。

2.4.2.3 激光建图

建图默认采用 Gmapping，输入为激光扫描与机器人运动估计，输出二维占据栅格地图。当前配置为：地图分辨率 0.05 m/格，地图发布更新间隔 1.0 s，激光有效使用距离 maxUrange 8.0 m，激光最大范围 maxRange 12.0 m，粒子数 30，平移触发量 0.1 m，旋转触发量 0.3 rad（约 17.2 °），初始地图范围 X、Y 均为 −20～20 m。5 cm 栅格是地图离散表示尺度，不等同于 5 cm 建图精度或定位精度。

激光雷达当前配置扫描角度 −85°～＋85 °，配置频率 10 Hz，无效距离用正无穷表达，配置距离区间 0.01～50 m；其中 50 m 为驱动配置值，不代表实际环境中的可靠测量能力。该链路对外使用约 170 ° 扫描扇区，不是持续全周覆盖，结合底盘横向运动能力，侧后方感知覆盖是实机验收的重要项目。建图操作按“确

认底盘、雷达、里程计和传感器 TF → 启动单一建图实例 → 人工控制机器人采集环境 → 观察墙体是否重影和地图是否明显扭曲 → 保存地图图像及 YAML 元数据 → 停止建图并切换定位导航模式 → 重新确认实际初始位姿”组织。地图保存在容器 /maps 下并通过宿主目录挂载持久化，地图图像与 YAML 应成套保存。

2.4.2.4 地图定位与路径规划

导航模式使用 map_server 加载已保存地图，由 AMCL 估计机器人位姿。当前主要配置为：运动模型 omni，最小/最大粒子数 2000/5000，激光模型 likelihood_field，最大使用光束数 60，平移更新阈值 0.25 m，旋转更新阈值 0.2 rad，地图坐标系 map，底盘参考坐标系 base_footprint。系统要求本次会话提供初始位姿并通过检查后才允许导航；RViz 的 2D Pose Estimate 用于告诉系统机器人实际所在位置和朝向，不是指定导航目标点。

路径规划采用 move_base 框架：全局规划负责从地图当前位置到目标的整体路线，局部规划根据当前运动状态和周围障碍输出速度。项目明确选择 DWAPlannerROS 作为局部规划器，TEB 配置存在但当前启动链路未启用；全局规划器在公共参数文件中未显式选定，按标准 Melodic move_base 默认行为应使用 navfn/NavfnROS，最终以运行时参数为准。DWA 当前主要配置为：X 方向最大速度 0.7 m/s，X 方向最小采样速度 0.01 m/s，Y 方向速度范围 −0.3～＋0.3 m/s，最大角速度 3.2 rad/s，X/Y 加速度限制 10 m/s²，角加速度限制 20 rad/s²，轨迹预测时间 1.0 s，控制循环频率 10 Hz，目标位置容差 0.1 m，目标朝向容差 0.1 rad（约 5.7°）。上述为规划参数，不是实测性能指标；目标容差 0.1 m 不代表真实世界定位精度达到 10 cm。

代价地图配置包括：全局代价地图参考系 map，局部代价地图参考系 odom，局部地图为随机器人移动的滚动窗口，窗口尺寸 2.5 m ×2.5 m，地图分辨率 0.05 m，障碍标记范围 3.0 m，射线清除范围 3.5 m，障碍观测源为激光 /scan，配置底盘轮廓约 0.305 m ×0.2456 m。轮廓尺寸来自配置坐标，不是本次测量的整机尺寸，也不能自动覆盖机械臂、线缆、支架或附加载荷。障碍层同时启用 marking、clearing 和 inf_is_valid；历史隔离软件实验表明，在“有限距离障碍出现后变为正无穷无回波”的特定场景下，启用 inf_is_valid 可将残留致命格从 9 个清除为

0 个，该结果支持特定场景下旧障碍清除逻辑得到改善，不支持“所有残留障碍均已解决”或“实车避障已验收”。

2.4.2.5 导航守卫与任务管理

导航守卫是本子系统相对于基础 ROS 导航配置的重点工程化设计。标准导航入口将真正的 move_base 动作接口与速度输出放到内部命名空间，用户目标先进入 navigation_guard，规划器产生的速度也要经过它再发布到 /cmd_vel。守卫检查是否提交本次有效初始位姿、地图/激光/AMCL 位姿是否可用、激光与 TF 是否足够新、初始位姿与估计结果是否一致、位姿协方差是否合理、激光与地图是否匹配，以及当前速度是否属于仍然有效的任务和规划尝试。直接向 /cmd_vel 发布的其他节点不会天然经过守卫，因此仍需控制运动指令的发布者。

首次定位确认采用相对严格的质量条件：激光匹配比例至少 0.65，抽样后有效回波数至少 30，射线冲突比例不高于 0.08，位置标准差不超过 0.35 m，航向标准差不超过 0.30 rad，持续满足条件 2 s。这里的“匹配比例”是程序定义的几何评分，不是定位正确概率；射线检查也不能证明在对称地图中的定位唯一性。当前导航入口默认 quality_interlock=false，即首次定位确认之后，位姿不确定性偏大、扫描与地图匹配下降、射线冲突、有效回波不足、AMCL 位姿过期、地图定位修正跳变等异常默认作为诊断告警，而不直接撤销导航任务；激光过期、TF 过期或缺失等传输和坐标链问题仍会触发停止速度输出、暂停规划等处理。该策略减少质量评分波动造成的频繁中断，但更依赖质量监测和现场验证。

守卫区分“任务结束”和“暂时不能继续执行”：出现可恢复故障时输出零速度、取消当前内部规划、保留任务；在时限内恢复稳定后重新规划。默认恢复参数为恢复稳定观察时间 1 s、暂态恢复时限 15 s、自动恢复次数上限 5 次、内部速度命令新鲜度 0.3 s。恢复时清理旧速度、使用规划尝试标识，并等待旧规划终止确认，避免过期命令或迟到结果影响新任务。标准守卫导航入口关闭了 move_base 自带恢复行为及清障旋转，系统暂停后重新规划不等于允许机器人自行旋转脱困。多点导航默认关闭，需要显式启用；起点书签保存地图关联位姿，辅助重复实验时恢复初始定位输入，并核对地图指纹、会话、请求时效及静止里程计，要求用户确认机器人已摆放到对应位置。书签的定位是帮助操作者重复提供初始位姿，不是机器人自动识别当前位置，也不是自动回到起点。

2.4.3 视觉感知部分

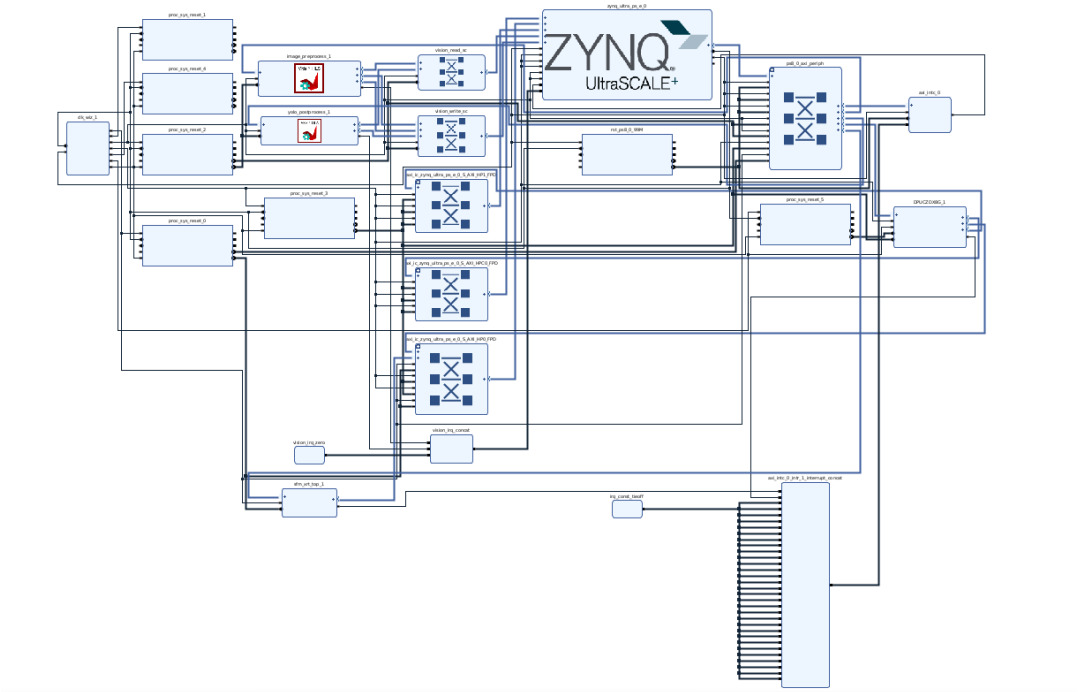


图 2-4 视觉加速平台计算单元组成

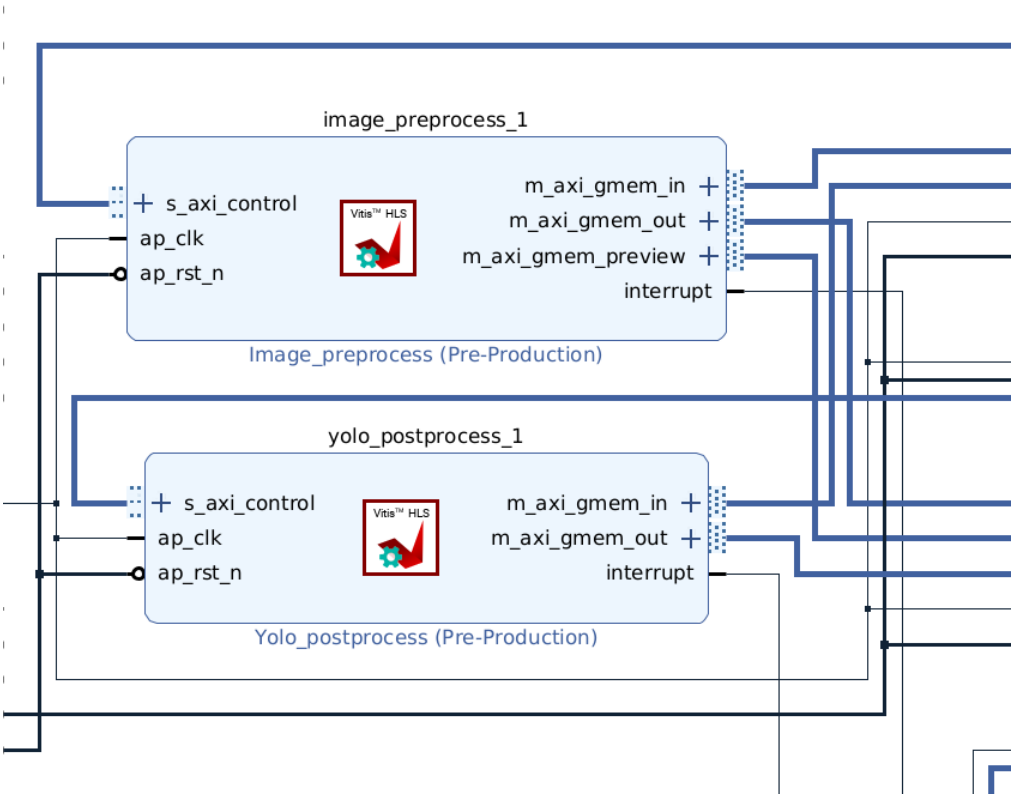


图 2-5 视觉加速平台数据流（一）

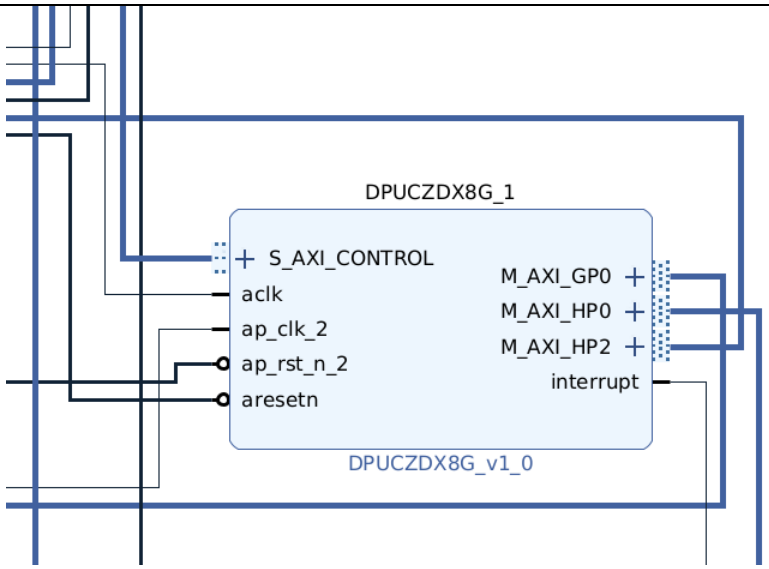


图 2-6 视觉加速平台数据流（二）

2.4.3.1 当前配置

本系统面向 AMD/Xilinx Kria KV260 构建 AI 视觉硬件加速平台，采用“ARM 处理系统负责控制、采集、调度和业务逻辑，FPGA 可编程逻辑负责图像预处理、神经网络推理和检测后处理”的软硬件协同架构。

当前系统集成三类核心计算单元：

图像预处理 HLS 核 image_preprocess_1：完成 BGR 图像读取、增强、双线性缩放、Letterbox、BGR→RGB、INT8 量化及张量输出，可选输出原分辨率增强预览。

DPU 推理核 DPUCZDX8G_1：执行与当前 DPU 架构和模型指纹匹配的神经网络推理。

YOLO 后处理 HLS 核 yolo_postprocess_1：完成 YOLOv4 三尺度输出解码、置信度过滤、类别选择和可选按类别 NMS。

另有独立 Softmax 核 sfm_xrt_top_1，但不属于 YOLO 检测框解码或 NMS 链路。工程目标是将推理前后的主要像素处理、量化和检测后处理纳入硬件加速，降低 ARM 端负担；CPU 占用、端到端 FPS、功耗和加速比仍需完整软件接入后的板端测试确认。

DPU 相关配置参数

项目	当前配置
目标平台	KV260，器件 xck26-sfvc784-2LV-c
工具版本	Vivado / Vitis / Vitis HLS 2022.1
DPU 实例	单个 DPUCZDX8G_1
DPU 架构	B3136，启用 URAM
DPU 时钟	标称 300 MHz aclk、600 MHz ap_clk_2
视觉核时钟	标称 200 MHz，平台元数据约 199.998 MHz
数据交换	AXI4 Memory-Mapped 访问 DDR，主机管理缓冲区和调度
控制接口	AXI4-Lite，支持轮询完成，硬件中断已连接
已有硬件产物	基础平台 XSA、最终链接 bitstream、dpu.xclbin

补充说明：基础平台不含 DPU；完整系统由后续 Vitis 链接加入 DPU 和 Softmax。当前 DPU 已由早期 B4096 调整为 B3136，不能直接假定旧 B4096 模型二进制仍兼容。

图 2-7 视觉感知数据流

2.4.3.2 数据流

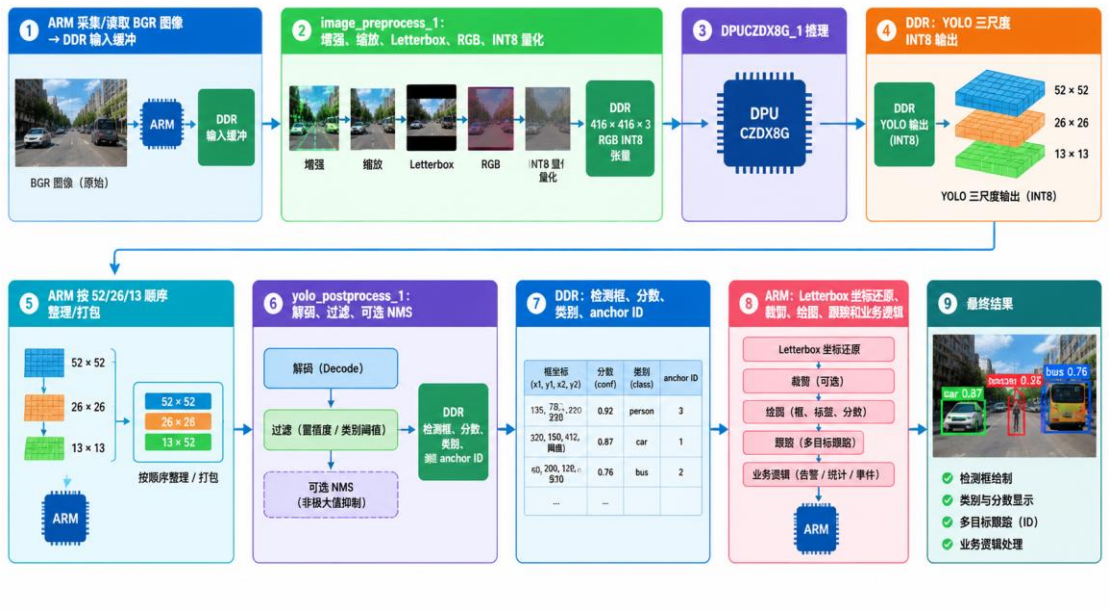


图 2-8 图像预处理 HLS 核结构

2.4.3.3 图像预处理加速核

image_preprocess 是按帧启动的内存映射 HLS IP，输入为未压缩 BGR888 图像，输出为模型张量、增强预览或二者同时输出。

图像预处理

项目	约束
输入格式	BGR888，每像素 B/G/R 各 8 位
输入尺寸	宽度 416~1920，高度 416~1080
行跨度	字节单位，16 字节对齐，至少 width×3，不超过 5760
张量输出	固定 [1,416,416,3]，NHWC、RGB、INT8
张量有效大小	519,168 字节
预览格式	原始输入分辨率的增强 BGR888
LUT 缓冲区	512 字节：256 字节颜色 LUT + 256 字节 INT8 量化 LUT
输出模式	TENSOR_ONLY、PREVIEW_ONLY、TENSOR_AND_PREVIEW
状态码	0 成功；1 尺寸错误；2 stride 错误；3 容量不足；4 模式错误；5 几何错误；6 饱和度错误

图 2-9 图像预处理数据流水线

主要处理流程为：DDR 读取 → 128 位数据解包 → 颜色 LUT/亮度对比度 → 定点饱和度 → 可选亮度直方图均衡 → 双线性缩放 → Letterbox 填边 → BGR→RGB → INT8 查表量化 → 张量打包写回；增强像素另一路生成原分辨率预览。

关键算法边界：

1、Letterbox 按等比例缩放并居中填边，填充值在量化前为 128；例如 640×480 缩放为 416×312，上下各填 52 行。

2、量化公式为 q(v)=clip_int8(round_even(v×2^(fix_point-8)))，对应本应用 /256 归一化约定，不能替换为 /255。

3、亮度/对比度通过主机生成颜色 LUT 实现；饱和度使用 Q12，4096 表示 1.0，范围 0～8192。

4、直方图均衡为全局亮度均衡，支持 EQ_OFF、EQ_CURRENT、EQ_PREVIOUS；当前帧模式读取图像两遍，上一帧模式单遍但有一帧滞后。

5、已有 OpenCV 对比覆盖中，量化后最大差异为 1 INT8 LSB；不宣称与所有 OpenCV 版本逐位一致，也不扩大为全部图像误差保证。

6、支持到 1920×1080 表示接受该尺寸，不代表已验证 1080p 实时帧率；小于 416 的输入不被当前硬件接受，应软件回退或另行实现。

HLS 实现采用 DATAFLOW、流 FIFO、双行 BRAM 缩放缓冲和 LUT 处

理；局部循环已有 II=1 调度证据，但整帧吞吐仍受统计、启动、访存和输出节拍限制。

2.4.3.4 YOLO 后处理加速核

yolo_postprocess 将 DPU 输出转换为检测结果，支持明确的 YOLOv4 416/COCO 风格 ABI，不是任意 YOLO 模型的通用后处理器。

YOLO 后处理

项目	要求/指标
模型输入尺寸	416×416
输出尺度及打包顺序	52、26、13
每尺度张量形状	[1,H,H,255]，NHWC、INT8
每网格 anchor 数	3
每 anchor 数据	4 坐标+1 objectness+80 类 logit，共 85 字节
类别选择	每个 anchor 仅保留最高分的一个类别
总 anchor 数	10,647
有效输入字节数	904,995
16 字节对齐最小输入分配	905,008 字节
查找表大小	5,376 个 u32，21,504 字节
候选框上限	256
输出最小容量	8,192 字节

图 2-10 YOLO 后处理 HLS 核结构

解码流程为：载入 sigmoid/宽高查找表 → 顺序扫描 anchor 的 85 个 INT8 数据 → 查表计算 objectness → 在 80 个有符号类别 logit 中选最大值 → 仅对选中类别做 sigmoid 并与 objectness 相乘 → 置信度过滤 → 解码中心坐标和框宽高 → 输出候选框或执行按类别贪心 NMS。

NMS 采用按类别贪心策略：每轮选择剩余最高分框，抑制同类别且 IoU 严格大于阈值的框；不同类别不互相抑制。硬件使用定点 LUT 和整数运算，不在线执行通用浮点 sigmoid/exp。

每条结果为 32 字节，包含 x1,y1,x2,y2（有符号 Q8，基于 416×416 Letterbox 图像的未裁剪坐标）、score（无符号 Q30）、class、anchor_id 和保留字段。

异常处理边界：

1、状态 0 成功，1 参数非法，2 候选溢出。

2、候选超过 256 时，硬件不返回可接受的截断结果；detection_count=0，

candidate_count 饱和在 256，主机必须对整帧执行原软件后处理。

3、enable_nms=0 时输出解码候选框，软件可继续执行 NMS；运行时关闭 NMS 不会移除已综合的 NMS 电路。

4、查表量化、定点坐标和 Q16 IoU 阈值可能改变接近阈值的决策；已有参考测试和 RTL 协同仿真不等于完整数据集精度验收。

2.4.3.5 平台集成与构建结果

表 2.5.1 地址、时钟与中断配置

项目	图像预处理	YOLO 后处理
BD 实例	image_preprocess_1	yolo_postprocess_1
AXI-Lite 基地址	0x86000000	0x86010000
地址窗口	64 KiB	64 KiB
时钟	clk_wiz_1/clk_out3	clk_wiz_1/clk_out3
核复位	proc_sys_reset_2/peripheral_aresetn	同左
完成中断	vision_irq_concat/In0 → PS IRQ1 bit 0	vision_irq_concat/In1 → PS IRQ1 bit 1

图 2-11 DPU 平台集成构建结果（一）

表 2.5.2 最终系统总资源

资源	使用量	总量	占比
LUT	80,948	117,120	69.12%
FF/CLB Registers	128,151	234,240	54.71%
BRAM Tile	119	144	82.64%
URAM	40	64	62.50%
DSP	642	1,248	51.44%
已占用 CLB 位置	14,623	14,640	99.88%

图 2-12 DPU 平台集成构建结果（二）

表 2.5.3 视觉核局部资源

核	LUT	Registers	DSP
image_preprocess_1	12,416	12,827	37
yolo_postprocess_1	4,371	7,204	25

图 2-13 DPU 平台集成构建结果（三）

2.4.3.6 软件接入与内存管理

2.4.3.6.1 初始化

加载同一构建版本的硬件产物与配套硬件描述，确认两个视觉核寄存器映射。

创建 DPU runner，读取模型输入输出形状、dtype、量化参数和 DPU 架构兼容信息。

确认预处理 RGB NHWC INT8 416×416 输入约定，以及后处理三尺度 80 类输出约定。

分配图像、LUT、张量、预览、后处理 staging 和结果缓冲区。

构造与模型实际 fix_point 对应的预处理量化 LUT 和后处理解码 LUT。

当前 DPU 为 B3136，应按当前 DPU 指纹/架构核查或重新编译模型。

2.4.3.6.2 每帧执行

采集或读取 BGR 图像，写入源缓冲区并同步。

设置预处理地址、几何、模式、帧 ID、配置 ID 和增强参数。

启动预处理，等待完成，检查状态、ID 和有效字节数。

将张量交给 DPU；若 runner 暂不支持共享设备缓冲区，先采用可验证的显式拷贝，不虚构零拷贝能力。

等待 DPU 完成，按 52/26/13 顺序整理 INT8 输出，写入后处理连续缓冲区。

同步后处理输入与 LUT，启动后处理并等待完成。

成功时读取有效记录；溢出时整帧执行原软件解码与 NMS；其他错误明确处理。

在 NMS 后还原 Letterbox 坐标并裁剪至原图，再执行绘图、跟踪或深度融合。

两个视觉核通过非一致性 HP 端口访问 DDR，主机必须使用真正的设备可访问物理缓冲区，检查 64 字节起始对齐、非重叠、stride、容量和地址范围，并在 CPU 写入后 flush/sync、硬件完成后 invalidate/sync。超时后不能立即回收缓冲区，应先确认旧任务停止或完成恢复。

2.4.4 机械臂控制部分

机械臂控制系统是本作品执行物品抓取、搬运、分类和归位任务的重要组成部分。系统配置两套机械臂，根据硬件结构和任务特点采用不同的控制方式：第一套以 LeRobot 框架和 ACT 动作策略为基础，面向工位物品的抓取与分拣；第二套以 Spartan-7 作为底层控制核心，通过 KV260 接入 ROS 系统，面向键盘操

作、规划轨迹执行和辅助作业。

两套机械臂由上层任务调度模块统一协调，与视觉感知、移动导航及执行反馈模块配合，形成从目标确认、动作生成到执行控制和结果检查的完整操作流程。系统总体设计如图 2-14 所示。

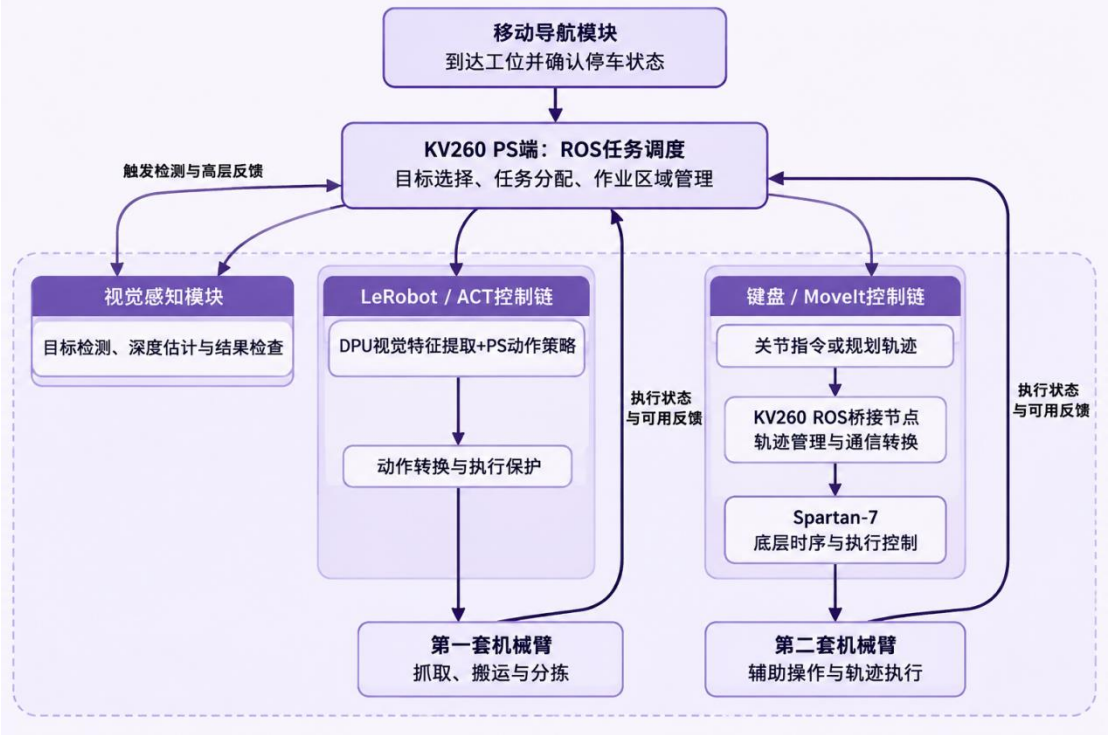


图 2-14 双机械臂控制系统总体设计

两套机械臂的功能定位与控制方式如表 2-1 所示。

表 2-1 两套机械臂的设计分工

对比项目	第一套机械臂	第二套机械臂
主要任务	工位物品抓取、搬运与分拣	手动调试、规划轨迹执行与辅助操作
上层控制方式	LeRobot 框架下的 ACT 动作策略	键盘操作、MoveIt 或任务调度
动作生成方式	根据图像与机器人状态预测动作块	生成关节指令或带时间信息的关节轨迹
主要计算平台	KV260 PS＋PL 中的 DPU	KV260 PS＋Spartan-7
底层执行路径	动作接口转换后下发至对应控制器	ROS 桥接节点向 Spartan-7 发送控制数据

视觉信息用途	策略观测、任务筛选与结果检查	几何目标生成、规划辅助与结果检查
重点验证内容	策略效果、量化误差、推理及控制延迟	运动学模型、通信协议、轨迹跟踪与反馈

2.4.4.1 基于 LeRobot 与 ACT 策略的机械臂控制

第一套机械臂采用 LeRobot 框架组织示范数据采集、策略训练与动作执行，并选 ACT 作为动作生成策略。系统通过摄像头采集工作区域图像，同时获取机械臂关节位置及夹爪状态，将两类信息按训练时的定义组织为策略输入。

ACT 通过学习示范数据中视觉观测、机器人状态与动作之间的关系，一次预测未来一段时间的动作序列。控制模块对预测结果进行反归一化、关节顺序映射及单位转换，再按照控制周期逐步下发，从而组织接近、抓取、抬升、搬运和放置等连续操作。

与传统几何控制不同，ACT 并不是直接将目标三维坐标转换为规划轨迹。YOLO 检测结果和深度信息主要用于目标筛选、任务触发及操作结果检查；若需要将其作为 ACT 的额外输入，则应在数据采集和训练阶段同步设计。

2.4.4.1.1 PS 与 DPU 协同推理

针对 ACT 的计算需求，系统拟采用“DPU 视觉特征提取＋PS 动作 Transformer”的划分方式。优先将视觉骨干部署到 DPU，PS 负责状态融合、Transformer 计算、动作处理和执行调度。其推理流程如图 2-15 所示。



图 2-15 ACT 在 KV260 上的软硬件协同推理设计

图中的 DPU 输出为保留空间信息的视觉特征图，而不是 YOLO 检测框或分类结果。视觉骨干必须使用对应 ACT 策略中的权重，并与 PS 侧网络保持一致。

2.4.4.1.2 计算模块分工

表 2-2 ACT 推理与控制模块的软硬件划分

模块	首版执行位置	主要职责	适配与验证重点
图像采集与观测同步	PS	组织相机图像和机器人状态	时间戳、相机顺序、数据新鲜度
图像预处理	PS；后续评估 HLS	缩放、颜色转换及归一化	与训练流程保持一致
ResNet 视觉骨干	DPU	提取空间视觉特征	算子支持、量化精度、特征误差
视觉特征投影	优先评估 DPU；必要时 PS	将视觉特征映射至策略维度	编译分图与边界开销
状态融合与 Transformer	PS	根据视觉和状态生成动作表示	CPU 耗时、线程配置及内存开销
动作后处理	PS	反归一化、关节映射、单位转换	输出语义与执行接口一致
动作块调度与执行保护	PS	周期下发、超时处理及人工接管	动作有效期、队列积压和反馈
底层执行	对应机械臂控制器	执行关节与夹爪指令	执行时间、反馈能力及通信可靠性

现有面向 YOLO 的预处理和检测后处理不能直接替代 ACT 流程。系统应先在 PS 端建立与训练一致的处理链，再根据分段计时结果决定是否引入 HLS 加速。

此外，动作块长度不等于实际控制频率。策略计算与指令发送需要分别调度，并处理动作块交接、观测延迟和预测结果过期等问题。实际加速效果应通过同一模型、同一输入条件下的 CPU 基线与混合部署对比验证，不能仅依据 DPU 子模块耗时判断整套系统的实时性。

2.4.4.2 基于 Spartan-7 和 ROS 的机械臂控制

第二套机械臂采用“上层指令生成—通信接口转换—底层时序执行”的分层架构。KV260 负责键盘指令处理、运动规划和轨迹执行管理，Spartan-7 根据执行机构的接口要求产生控制时序，驱动各关节及末端执行器完成动作。

ROS 桥接节点负责连接两层系统。它接收上层关节指令或规划轨迹，将其转换为约定的通信数据，并对硬件能够提供的位置、运行状态和故障信息进行整理和发布。接口设计应明确关节编号、目标位置、执行时间、指令序号及校验规则，保证指令和反馈能够正确关联。

键盘控制主要用于单关节调试、夹爪操作和基础动作验证。末端位姿调整需要运动学求解支持，不能仅依靠底层时序控制完成。

在建立机械臂 URDF 模型、关节限位、运动学参数和碰撞模型后，可进一步接入 MoveIt。MoveIt 负责生成带时间信息的关节轨迹，ROS 桥接节点负责执行管理，Spartan-7 负责底层输出。由此明确：

KV260 软件层：完成目标处理、运动学求解、碰撞检查和轨迹规划。

Spartan-7 硬件层：完成指令解析及对应的底层控制时序。

反馈与保护层：监测实际执行情况，处理越限、超时和异常状态。

目标位置与实际反馈应分别保存，不能将“指令已发送”直接视为“机械臂已到位”。

2.4.4.3 机械臂与视觉系统的协同控制

视觉系统通过 YOLOv4 获得目标类别、检测框和置信度，并结合经过配准的深度图像估计物体位置。几何信息处理流程为：

目标检测 → 有效深度选取 → 相机内参反投影 → 相机坐标系三维位置 → 外参变换 → 机械臂基座坐标系目标位置。

其中，目标位置并不等同于完整抓取位姿。抓取姿态、接近方向和夹爪开度仍需结合物体形状、摆放状态、末端执行器结构及障碍物情况确定。

两套机械臂对视觉信息采用不同的使用方式：

ACT 控制链：图像作为策略观测，检测与深度信息辅助任务选择、操作条件检查和结果确认。

MoveIt 控制链：几何信息用于生成目标位姿，再由运动学求解和规划模块产

生关节轨迹。

为减少无效操作，系统设计在任务执行前、执行中和执行后分别进行检查，如表 2-3 所示。

表 2-3 视觉与机械臂协同控制检查项

阶段	检查内容	正常处理	异常处理
目标确认	类别、置信度及连续观测稳定性	确认候选目标	重新观察或人工确认
深度处理	深度有效性、配准结果及数据时效	估计目标空间位置	丢弃无效数据，不触发抓取
操作准备	工作空间、可达性及共享区域占用	分配机械臂与作业区域	更换目标、等待或退出任务
动作生成	策略输出或规划轨迹的有效性	进入执行队列	拒绝异常输出并记录原因
动作执行	限位、动作变化幅度、通信及反馈	按周期执行	进入经过验证的安全处理流程
结果检查	原工位与目标容器中的物品状态	标记任务完成	重新确认、有限重试或人工处理

ACT 动作预测不能替代碰撞检查和执行保护；MoveIt 生成的轨迹同样需要结合实际反馈判断执行情况。安全检查应独立于模型推理运行，避免推理延迟阻塞异常处理。

2.4.4.4 机械臂任务调度与结果确认

机械臂任务由 ROS 任务调度模块统一管理。移动小车到达工位并满足停车及作业条件后，调度模块根据目标类别、位置、分拣规则和机械臂可用状态选择执行单元。每次任务关联目标物品、目标容器、执行机械臂和任务编号，以避免视觉结果、动作指令及反馈发生错配。

任务闭环如图 2-16 所示。

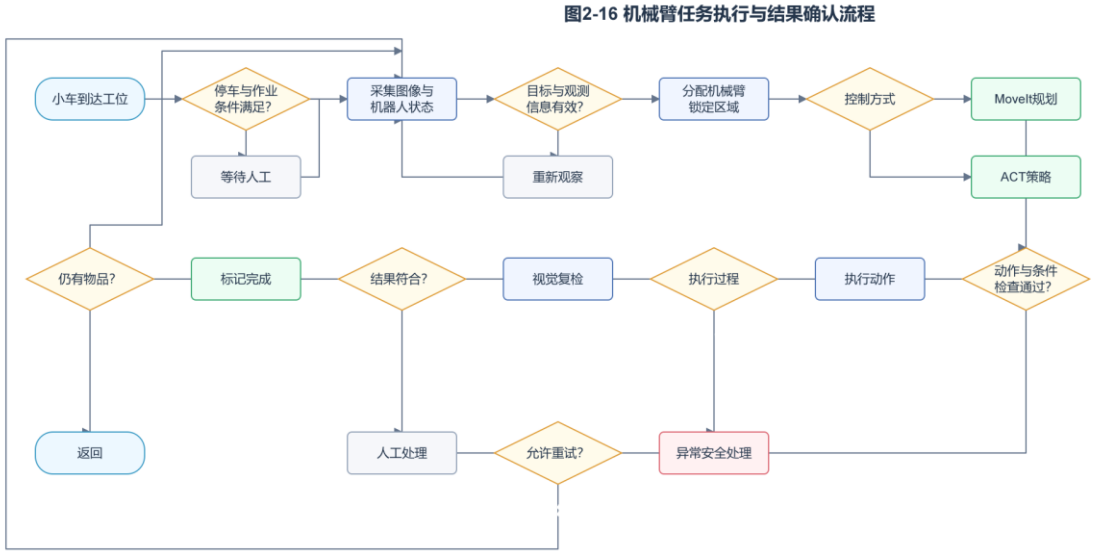


图 2-16 机械臂任务执行与结果确认流程

两套机械臂共享作业空间时，系统优先采用区域互斥和顺序作业，避免同时进入存在碰撞风险的区域。在完成空间关系、时序协调和碰撞风险验证后，再扩展并行协同操作。

对于 ACT 控制链，策略模块负责生成动作块，执行模块按照控制周期读取有效动作。新动作块接入时需要检查观测时间、预测时域和当前执行进度，避免将积压或过期动作继续下发。对于 Spartan-7 控制链，轨迹执行管理需要关注时间参数、通信状态和底层反馈，不能仅以发送结束作为任务完成条件。

抓取与放置后，视觉系统再次检查原工位和目标容器，并结合可获得的夹爪与关节反馈确认结果。对抓取失败、物品掉落、放置偏差或结果不明确的情况，应先重新评估现场，再决定有限重试或人工处理，避免在错误信息下反复操作。

通过上述设计，系统将学习型动作策略、传统运动规划和硬件底层控制结合起来，形成具有明确计算分工、执行接口和反馈机制的双机械臂架构。后续以端到端延迟、动作精度、任务成功率及异常处理有效性作为验证依据，逐步确认软硬件协同加速和双机械臂协作的实际效果。

2.4.5 通信部分

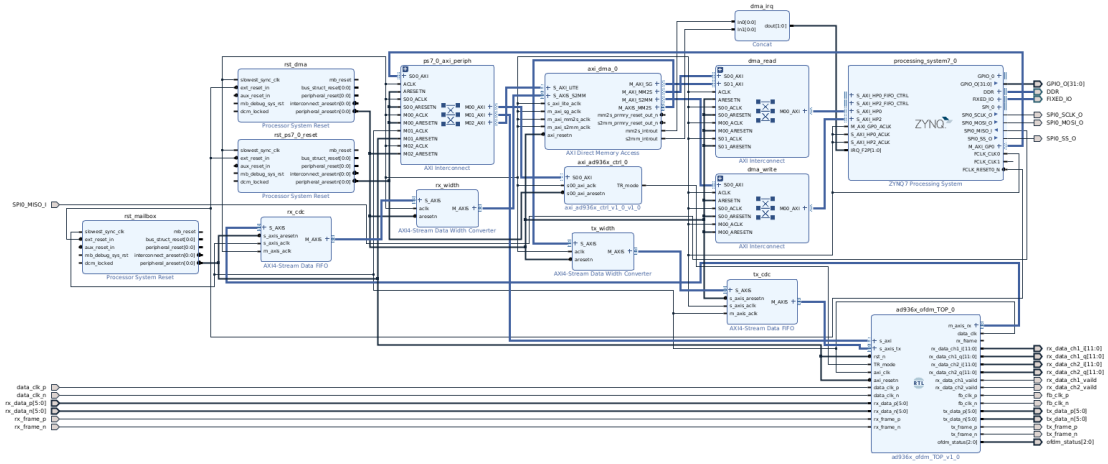


图 2-17 OFDM 收发链路结构

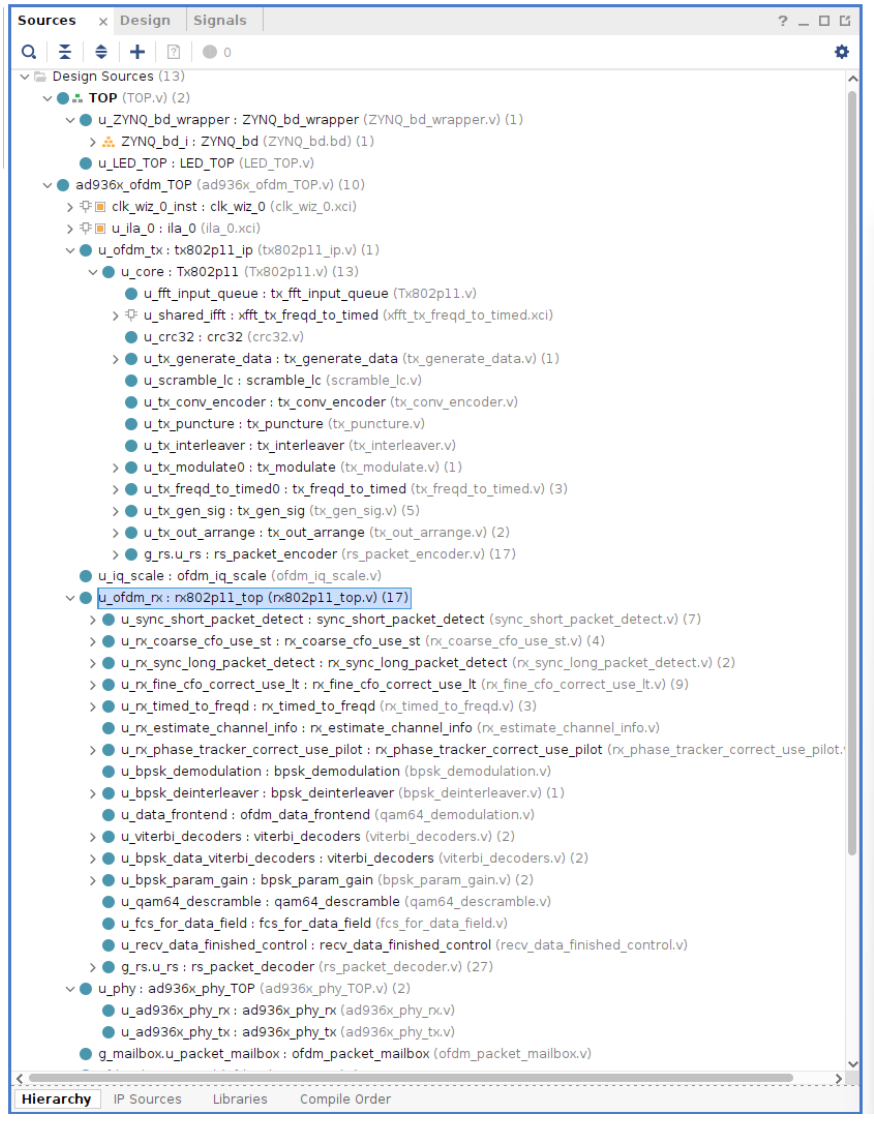


图 2-18 OFDM 数据流与协议分层

2.4.5.1 通信平台定位

本通信平台基于 Zynq-7020 与 AD9361 构建 LWIP+OFDM 软件无线电数据通道，主要作用是验证 OFDM 收发链路以及 “上位机/KV260 → TCP/UART → PS/DMA → OFDM TX → RF → OFDM RX → DMA/PS → 上位机/KV260” 的数据通路是否正确。它重点验证包格式、DMA 搬运、OFDM 调制解调、CRC/FCS 校验、RS 纠错、队列与 ACK 机制能否形成完整闭环，从而确保发射端数据正确无遗漏。

2.4.5.2 系统组成与分工

表 2-4 OFDM 通信平台组成与分工

部分	主要功能
Zynq-7020 PS	AD9361 初始化与控制；SG-DMA 描述符和 DDR 缓冲区管理；cache 维护；lwIP TCP 服务；UART 业务协议；分片、ACK、重试和状态统计
Zynq-7020 PL	AD936x LVDS 接口；OFDM TX/RX PHY；CRC32/FCS；RS(255,239) 编解码与字节交织；TX/RX 包缓冲；AXI-Stream/AXI-Lite 接口
AD9361	RF 收发、本振/带宽/增益配置、ADC/DAC 与 LVDS 样本接口
KV260/上位机测试端	发送测试数据或图像分片，接收回传数据，执行 CRC、逐字节一致性和分片重组检查

KV260 或上位机作为数据源和校验端，通过网口/串口与通信平台连接；通信平台内部由 PS 完成协议与 DMA 管理，由 PL 完成 OFDM 基带处理和包校验，由 AD9361 完成射频收发。

2.4.5.3 收发环路验证流程

发送方向：

上位机/KV260 将测试文本或图像分片封装为业务包，经 TCP 或 UART 送入 Zynq-7020 PS；PS 完成协议校验、分片和队列管理后，由 SG-DMA 将数据从 DDR 搬运至 PL；PL 的 TX 适配器在 payload 前加入 16 字节本地元数据，随后完成 CRC32/FCS、可选 RS(255,239) 编码与字节交织、扰码、卷积编码、打孔、OFDM 比特交织、星座映射、IFFT、循环前缀和 I/Q 缩放，再经 AD9361 LVDS TX 与射频前端发出。

接收方向：AD9361 将 RF 信号下变频并输出 LVDS 样本；PL 完成前导检

测、符号同步、频偏校正、去 CP、FFT、信道估计与均衡、导频跟踪、SIGNAL 解析、解调、去交织、Viterbi 解码、解扰、RS 去交织/纠错和 CRC/FCS 检查。只有长度有效且 FCS 正确的数据包才进入 RX 队列，经 SG-DMA 写入 DDR，再由 PS 通过 TCP/UART 返回上位机/KV260。上位机对回传 payload 执行 CRC32、逐字节比较和分片重组检查，从而判断收发环路是否正确。

该流程的验收判据不是“TCP 连接成功”或“sendall 成功”，而是：RX 实际返回 payload 与发送内容一致、CRC/FCS 通过、ACK 窗口清空、分片重组完整。单字节 UART 记录仅表示本地 DMA 入队状态，不能单独作为无线回环成功依据。

2.4.5.4 关键设计

（1）OFDM 与速率配置。 PHY 采用 IEEE 802.11a 风格 OFDM，支持 6/9/12/18/24/36/48/54 Mbps，调制方式包括 BPSK、QPSK、16-QAM、64-QAM，卷积码率包括 1/2、2/3、3/4。TX 顶层缓存完整用户包后连续回放，输入允许间隙，但发射核心不接收任意重叠事务。

（2）CRC 与 RS 纠错。 用户 payload 长度为 1..1024 字节，CRC32/FCS 作为最终包完整性检查保留。RS 采用缩短 RS(255,239)、GF(256)、本原多项式 0x11d，每码字 16 个校验字节，可纠正至多 8 个符号/码字错误。当前 RTL 默认 RS_ENABLE=1、RS_INTERLEAVE=1，将 payload+CRC 平衡拆分为 D 个连续数据块，分别 RS 编码后按列交织；每个码字仍只有 8 字节纠错能力，连续突发保证为 8D 字节，该指标是 Viterbi 解码/解扰后字节流条件下的保证，不是 RF 采样级抗干扰指标。RS/交织参数为编译期配置，没有空中自动协商，收发两端必须一致。

（3）DMA 与包接口。 DMA TX 输入为 AXI-Stream 包，开头 16 字节为本地元数据：payload 长度、OFDM 速率值、上采样配置、cookie 和保留字段；偏移 16 起为 1..1024 字节无线 payload。PS 固件配置 128 个 TX/RX 描述符、2048 字节 slot、至多 4 个前置 outstanding 包，并在 CPU 与 DMA 间执行 D-cache flush/invalidate。RX 端在 FCS 到达前把解调字节暂存在包 RAM，只有 FCS 有效、长度匹配且队列有可用 slot 时才提交 DMA；不满足条件时增加 drop/overflow/timeout 计数。

（4）TCP/UART 业务封装。 UART 采用 115200、8N1，请求为 A5 5A| 长度 |OF 包，响应为 5A A5| 长度 | 记录。TCP 监听端口 5001，主机业务包头长 24 字节，包含 magic OF、版本、类型、通道、session、message、分片序号、分片总数、payload 长度和总长度；图像按 1000 字节分片，服务端用 bitmap 防止重复写入，完成后对完整对象执行 CRC32 检查。上述协议用于回环测试和数据通路验证，远程交互应用作为后续扩展。

2.5 软硬件划分依据与接口设计

2.5.1 软硬件划分总体原则

三个工程并非采用同一种加速架构：

OFDM：硬件处理连续采样和物理层，软件管理协议与可靠传输。

DPU 视觉：硬件执行主要计算，软件通过 DDR 缓冲组织帧级流水线。

导航：CPU 运行定位、规划和状态机，通过串口连接底盘控制板；未发现导航算法迁入 PL 的实现证据。

表 2-5 软硬件划分总体原则

划分依据	更适合硬件承担的工作	更适合软件承担的工作	工程中的体现
时序确定性	连续采样、固定节拍、流水计算	毫秒级调度、超时与任务管理	OFDM PHY 在 PL；重传策略在 PS
计算规律性	重复算术、像素运算、神经网络计算	分支复杂、动态状态、多协议交互	HLS 预后处理、DPU 推理；CPU 负责调度
数据搬运成本	连续流或批量访问	描述符、缓冲区生命周期管理	OFDM 采用 SG-DMA；视觉采用 CMA 缓冲
算法可变性	边界稳定、接口明确的计算核	参数调整、任务策略、界面与诊断	图像核保持 ABI；导航策略保留软件实现
资源与性能权衡	用流水、DSP、BRAM 换取吞吐或时序	避免低收益的硬件复杂化	HLS 按布线结果调整乘法实现
异常处理	检测溢出、坏包、非法参数并返回状态	决定重试、整帧回退、停止或恢复	后处理溢出回退 CPU；导航状态机处理恢复

2.5.2 各工程功能划分

表 2-6 各工程功能划分

工程	执行位置	主要职责	划分理由
OFDM	AD936x 射频器件	射频收发及数字 I/Q 接口	模拟射频与数字基带分层
OFDM	Zynq PL	OFDM 调制解调、同步、FFT/IFFT、均衡、编译码、RS/交织、FCS、包缓存	连续采样和并行流水处理，不依赖 CPU 逐样本调度
OFDM	Zynq PS	射频配置、DMA 管理、缓存维护、lwIP、UART、ACK/重试、分片及状态统计	协议状态复杂，便于修改、诊断与恢复
OFDM	主机软件	数据提交、图像重组、逐字节/CRC/SHA-256 核验、显示	文件和交互逻辑不占用 PL 资源
视觉	HLS 前处理核	缩放、填充、颜色处理、量化及可选预览增强	像素计算规律，输入输出格式可固定
视觉	DPU	编译模型中的 DPU 子图推理	将主要神经网络计算交给专用加速器
视觉	HLS 后处理核	YOLO 解码、候选筛选、NMS	将重复定点计算下沉硬件
视觉	ARM/Linux 软件	相机采集、缓冲管理、VART 调度、深度配对、CPU 回退、ROS2 发布和 GUI	对接系统接口并处理动态策略与异常
导航	KV260 CPU / ROS1	AMCL、move_base、定位确认、任务管理、速度门控、书签和诊断	地图、传感器和任务状态耦合复杂，适合软件迭代
导航	底盘控制板及外设	接收运动指令，提供速度、IMU 等反馈	将上层导航与底层执行分离；底盘内部控制细节未在本次核验范围内

2.5.3 OFDM：流式数据面与软件协议控制

2.5.3.1 数据流

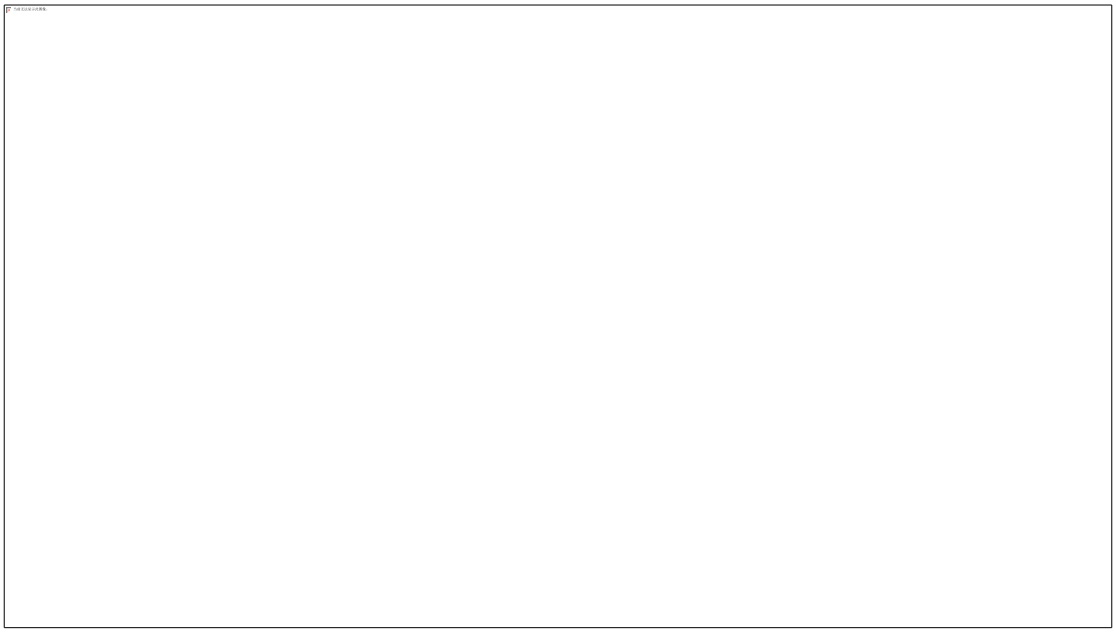


图 2-19 OFDM 数据流

2.5.3.2 接口设计

表 2-7 OFDM 流式数据面接口

接口边界	接口形式	数据与控制约定
主机 ↔ PS	TCP，监听端 5001	长度前缀组织记录；同一连接同时上传和接收，处理 TCP 无消息边界的问题
主机 ↔ PS	UART，1152008N1	使用帧头、长度字段；“本地入队成功”与“收到匹配 RX 数据”分开判定
PS ↔ DMA	DDR 缓冲＋SG 描述符	当前源码为 TX/RX 各 128 个描述符、2048 字节 slot；描述符和缓冲 64 字节对齐
DMA ↔ PL 包适配器	AXI-Stream	包适配器侧为 8 位字节流；使用 valid/ready 握手和 last 包尾，BD 承担位宽/时钟域转换
PS ↔ PL 控制器	AXI-Lite	enable、版本识别、最大包长、提交/完成/丢弃/超时等计数
PL ↔ AD936x	数字 I/Q 数据接口	样本流与包流分开；不能把包级背压能力理解为 RF 采样端可任意暂停

两层包头必须区分

表 2-8 两层包头格式区分

层次	格式	用途
本地 DMA TX 头	16 字节元数据＋无线 payload	长度、速率、上采样配置、cookie 及保留字段；由 TX 适配器解析，不作为这 16 字节原样发射
无线业务包	24 字节 OF 协议头＋最多 1000 字节业务数据	类型、通道、session、message、分片序号/总数、有效长度及总长度
PHY 处理	对无线包执行编译码、RS/交织及 FCS 等处理	与主机文件级校验分层

DMA 头中长度和 cookie 按小端编码；无线业务包最大 1024 字节。因此，“无线包最大 1024 字节”不等于“单片业务数据可达 1024 字节”。

2.5.3.3 正确性与故障处理

先校验、后提交： RX 先缓存，只有 FCS、包长及队列容量满足条件才进入 DMA。

缓存一致性： CPU 提交数据前 flush，读取 DMA 结果时 invalidate。

有界队列： 限制在途包，不能依靠无限缓冲掩盖处理能力不足。

分层完成语义： DMA 完成、PHY 发送完成、收到 ACK、主机文件完整分别判断。

跨版本一致性： bitstream、XSA、固件与协议必须匹配；历史 mailbox 模式不能与 DMA 模式混用。

源码还记录过一个典型接口缺陷：DMA 配置启用了实际不存在的状态/控制流，导致数据已写入 DDR 但 RX 描述符不完成。这类问题必须修正硬件配置，不能仅重编译软件解决。

2.5.4 DPU/HLS 视觉：DDR 共享缓冲与帧级调度

2.5.4.1 数据流

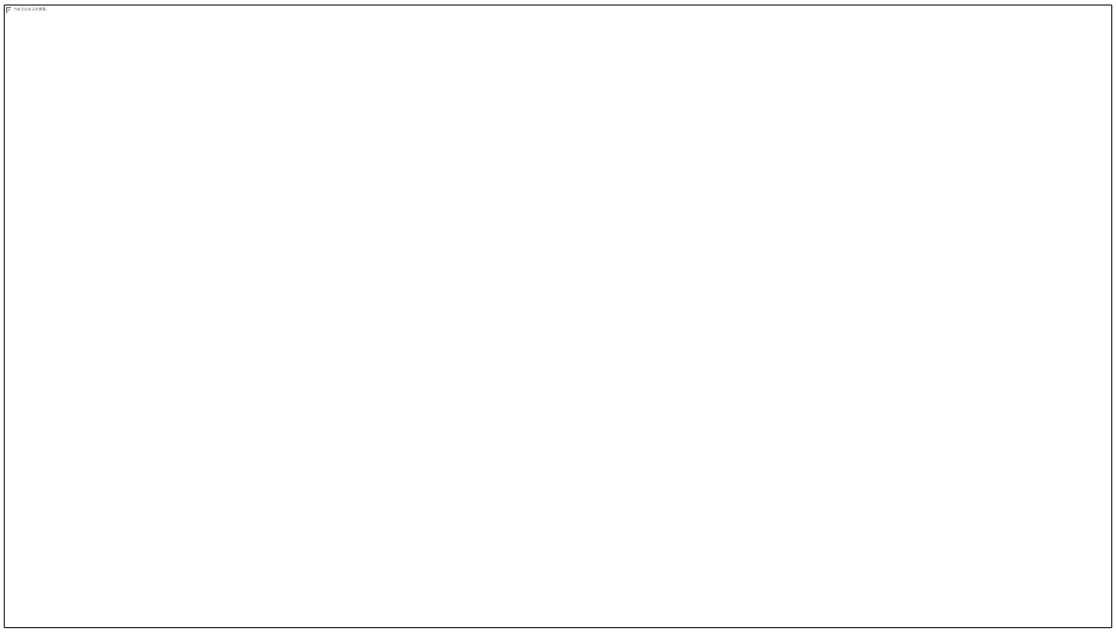


图 2-20 DPU/HLS 视觉帧级调度数据流

2.5.4.2 接口设计

表 2-9 DPU/HLS 视觉接口约定

接口边界	实现方式	主要约定
CPU ↔ HLS 控制	AXI-Lite	写入设备地址、尺寸、stride、容量及参数；启动后等待完成并读取返回状态
HLS ↔ DDR	m_axi	前处理源图像、张量及预览按 128 位字访问；后处理输入 128 位、检测输出 256 位
CPU ↔ DPU	VART runner	execute_async()提交，wait()确认完成；张量形状和量化参数匹配编译模型
运行时 ↔ HLS 寄存器	从 HWH 解析偏移	避免将另一版 IP 的寄存器地址直接套用
视觉程序 ↔ ROS2	检测主题，String 承载 JSON	/yolov4_b3136/detections；BEST_EFFORT、depth=1，偏向最新结果而非可靠历史存档

前处理输入输出契约

表 2-10 图像前处理输入输出契约

项目	当前约定
源图像	BGR 字节顺序；显式提供 width、height、stride 和 capacity
最大源尺寸	1920×1080
模型输入	1×416×416×3、INT8、RGB 顺序
当前 640×480 路径	缩放为 416×312，上下各填充 52 行
缓冲要求	不同用途缓冲互不重叠，64 字节对齐；传设备地址，不传普通虚拟地址
参数一致性	使用 frame_id、config_id 及完成回读，防止帧与配置错配
控制时序	禁用 auto-restart；前一帧 ap_done 前不能改寄存器或相关内存参数
返回检查	状态、ABI 版本、完成帧号/配置号及输出字节数共同校验

后处理输入输出契约

表 2-11 YOLO 后处理输入输出契约

项目	当前约定
输入尺度顺序	52、26、13
布局	NHWC；每网格 3 个 anchor，每个 anchor 85 个 INT8 值
候选容量	最多 256 个
检测记录	每条 32 字节：4 个坐标、分数、类别、anchor 索引及保留字段
定点语义	坐标为带符号 Q8，分数 Q30；置信度阈值 Q30，NMS 阈值 Q16
返回状态	0 正常、1 非法参数、2 候选溢出
溢出策略	整帧 CPU 解码和 NMS，不能发布被截断的硬件结果

该后处理接口针对当前模型布局，不是任意 YOLO 版本都能直接接入的通用接口。

2.5.4.3 调度与缓冲所有权

当前默认采用 1 个 VART runner、2 个帧槽；采集等待队列和结果队列保留最新帧。

其依据不是“线程越多越快”，而是前述实验显示：

双 runner 吞吐增益较小。

减少帧槽显著降低排队延迟。

有界缓冲能明确内存占用和停止条件。

缓冲生命周期遵循：

CPU 写入 → flush → 加速器使用 ▼ 确认完成 ▼ invalidate → CPU 读取

需特别区分两条边界：

VART 内部仍存在 host/device 搬运，不能声称全链路零拷贝。

超时不等于硬件已停止。 若加速器可能继续访问内存，必须保留缓冲与资源锁，不能直接释放或重载 overlay。

2.5.5 X3plus/KV260 导航：软件任务控制与底盘执行分离

2.5.5.1 数据流

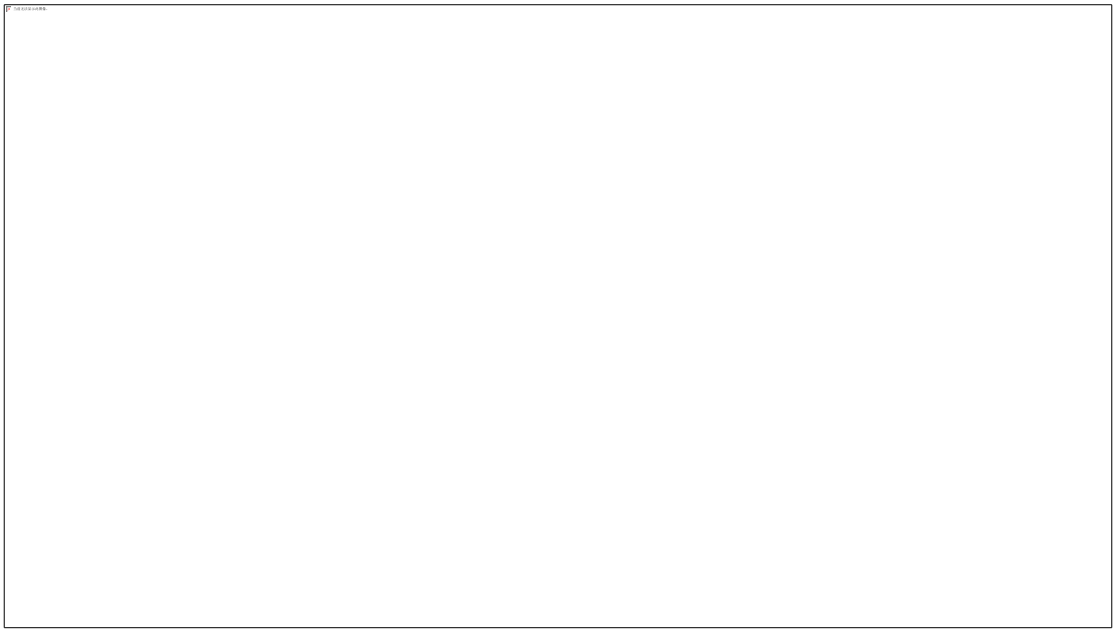


图 2-21 缓冲生命周期

2.5.5.2 消息与服务接口

表 2-12 导航子系统消息与服务接口

接口	数据类型/形式	设计作用
/scan	LaserScan	激光观测，检查数据及时间有效性
/map	OccupancyGrid	定位参考；地图变化触发重新确认
/amcl_pose、/initialpose	PoseWithCovarianceStamped	区分估计位姿与用户初始化意图
/move_base_simple/goal	PoseStamped	接收简化导航目标
公开/move_base	MoveBaseAction	对上层提供统一任务接口
内部/navigation_internal/move_base	MoveBaseAction	与实际规划执行分离，便于取消、恢复和隔离旧回调
内部速度 → /cmd_vel	Twist	门控审核后才向底盘输出；队列长度为 1
/navigation/ready、/navigation/localization_status	Bool、JSON String	分别表达可用状态与详细诊断
/navigation_guard/stop	Trigger 服务	取消任务，但保留当前定位确认
/navigation_guard/reset	Trigger 服务	清除定位确认，要求重新设置初始位姿
驱动 ↔ 底盘	Rosmaster 串口，115200	将 vx、vy、角速度编码下发，并接收反馈

2.5.5.3 安全与版本语义

目标、定位和执行分开编号： 通过 task/localization 等 epoch 避免旧执行回调污染新任务。

初始位姿不是可信真值： 必须经过地图和传感器一致性检查。

stop 与 reset 分开： 停止运动不应无条件要求用户重新定位。

驱动看门狗： 当前源码在约 0.5 秒无新速度命令时下发零速度，检查线程约每 0.1 秒运行一次。它是软件停止请求，不是物理制动时间保证。

2.5.6 机械臂部分

机械臂控制涉及视觉特征提取、动作策略推理、运动规划、指令调度和底层执行等不同类型的任务。系统根据计算特点、时序要求及接口复杂度，将相关功能划分至 KV260 的 PS 端、PL 端 DPU 以及 Spartan-7 控制板。其中，第一套机械臂采用 LeRobot 框架和 ACT 动作策略，拟通过 DPU 加速视觉特征提取，由 PS 完成 Transformer 计算和动作调度；第二套机械臂通过 ROS 接收键盘或运动规划指令，由 PS 完成轨迹管理和通信转换，Spartan-7 负责底层执行时序。两条控制链在上层使用统一的任务管理与状态反馈机制，在底层保留各自的硬件适配接口。

2.5.6.1 机械臂软硬件划分依据

2.5.6.1.1 按计算特点划分视觉与策略计算

ACT 的视觉骨干包含大量卷积运算，适合作为 DPU 部署的优先对象。系统拟将经过量化和编译验证的视觉骨干部署至 DPU，并评估是否合并后续特征投影层。DPU 输出保留空间结构的视觉特征，由 PS 继续完成状态融合、Transformer 计算和动作输出。

Transformer 包含注意力、归一化及张量重组等计算，不能仅因 DPU 支持部分矩阵运算就认为整个网络均可部署。首版将其保留在 PS，减少复杂分图和频繁数据交接；后续根据实测瓶颈再评估进一步加速。

2.5.6.1.2 按时序要求划分规划与执行控制

任务管理、运动规划及策略推理具有较强的软件灵活性需求，安排在 PS 端执行。Spartan-7 负责第二套机械臂执行机构所需的底层控制时序，将高层任务处理与底层输出分离。

2.5.6.1.3 按动作语义区分两条控制链

ACT 控制链根据图像与机器人状态预测动作序列；传统规划控制链根据目标位姿、运动学模型和环境信息生成关节轨迹。两者可以共享任务调度和执行保护机制，但不能直接混用输入输出：

ACT 动作必须按训练时的关节顺序、单位及归一化规则解释。

MoveIt 轨迹必须按机器人模型、关节名称和时间参数解释。

目标检测框、目标三维位置不能直接替代 ACT 的视觉输入，也不等同于完整抓取位姿。

2.5.6.1.4 将执行保护独立于推理与规划

模型输出或规划结果均需经过执行条件检查。指令有效性、关节限位、动作变化幅度、通信超时及人工接管等功能由独立控制逻辑管理，避免模型推理阻塞异常处理。

2.5.6.2 机械臂模块分工

表 2-13 机械臂控制模块的软硬件划分

功能模块	执行位置	主要职责	划分依据与验证重点
示范数据处理与策略训练	离线训练环境	数据整理、ACT 训练及模型评估	与板端实时控制分离
图像与机器人状态组织	KV260 PS	时间同步、输入检查和数据归一化	需要灵活的软件处理
ACT 视觉骨干	KV260 PL 中的 DPU，拟部署	提取空间视觉特征	卷积密集；需验证编译支持与量化误差
图像预处理	首版 PS，后续评估 HLS	颜色转换、缩放及归一化	先保证训练与部署一致，再评估加速
ACT Transformer 与动作输出	KV260 PS	融合视觉和状态，预测动作块	首版保留浮点计算，降低部署复杂度
动作后处理与周期调度	KV260 PS	反归一化、接口映射、有效动作下发	必须保持动作语义和时间一致性
键盘控制与 MoveIt 规划	KV260 PS，按接口接入	操作指令处理、运动学求解及轨迹生成	软件交互和规划逻辑复杂
ROS 与控制板通信桥接	KV260 PS	指令转换、传输管理和反馈发布	统一上层接口、隔离底层差异

第二套机械臂底层控制	Spartan-7，按硬件接口实现	解析控制数据并产生执行时序	面向执行机构的确定性输出
任务协调与执行保护	PS 监督；底层按能力设置保护	作业区域管理、超时处理及异常退出	不依赖单一模型或上位机状态

2.5.6.3 机械臂控制与数据接口设计

2.5.6.3.1 视觉及策略输入接口

视觉输入明确相机标识、图像尺寸、颜色通道顺序、预处理配置和采集时间。机器人状态明确关节顺序、单位、零点定义及反馈时间。深度估计和目标检测结果主要用于任务筛选、几何规划及结果检查。

2.5.6.3.2 PS 与 DPU 推理接口

PS 与 DPU 之间通过运行时管理的张量缓冲区传递数据。接口明确输入输出尺寸、布局、数据类型及量化参数，并完成必要的同步和缓存一致性处理。PS 负责提交推理任务、检查完成状态，并将 DPU 输出转换为后续 Transformer 需要的格式。

2.5.6.3.3 动作与轨迹执行接口

ACT 输出先恢复到执行接口使用的动作单位，再进行关节映射、范围检查和周期调度。动作块应携带对应观测时间、任务编号和有效时间范围，避免执行过期预测。MoveIt 规划结果则需保留关节名称、轨迹点和时间参数。执行层负责将其转换为控制板能够执行的指令。

2.5.6.3.4 ROS 与 Spartan-7 通信接口

该接口将 ROS 侧的关节指令转换为控制板协议。覆盖指令序号、关节目标、执行时间、使能状态和校验信息；反馈侧应区分接收确认、执行状态、实际位置及故障信息。

表 2-14 机械臂主要接口及约束

接口边界	主要数据	必须约定的内容	异常处理
相机 / 状态采集 → 策略输入	图像、关节状态、夹爪状态	时间戳、数据顺序、单位及预处理规则	拒绝过期或不完整观测

PS → DPU	量化输入张量	尺寸、布局、量化参数、缓冲区生命周期	提交失败或超时不得使用无效输出
DPU → PS	空间视觉特征	特征形状、输出量化参数、观测关联	检查完成状态及输出有效性
ACT → 执行管理	预测动作块	动作定义、控制周期、有效时间范围	丢弃过期或异常动作
MoveIt → 执行管理	关节轨迹	关节名称、位置及时间参数	拒绝越限或无法执行的轨迹
ROS 桥接 → Spartan-7	底层控制数据	指令序号、目标值、执行时间和校验	超时、错序或异常帧进入规定处理流程
控制器 → ROS	状态与可用反馈	区分目标值、实测值和完成状态	反馈失效时不得默认执行成功
任务调度 → 两套机械臂	任务分配与作业授权	任务编号、执行对象及共享区域占用	冲突任务等待或退出

2.5.6.4 已有接口的复用范围

表 2-15 现有 X3plus 机械臂接口与复用边界

接口	类型	现有用途	接入新控制链时的注意事项
TargetAngle	yahboomcar_msgs/ArmJoint 话题	下发单关节或多关节角度指令	转换动作单位、顺序和执行时间；防止旧指令积压
ArmAngleUpdate	yahboomcar_msgs/ArmJoint 话题	通知软件侧角度更新	不能作为实际到位确认
CurrentAngle	yahboomcar_msgs/RobotArmArray 服务	请求底层读取舵机角度	检查读取有效性、耗时与数据新鲜度
joint_states	sensor_msgs/JointState 话题	发布模型关节状态	当前主要由缓存角度生成，不应直接当作连续实测反馈

现有驱动中的较长订阅队列、重复下发和固定等待，需要在策略接入前重新评估。驱动初始化时还会发送预设姿态，因此部署时应将“软件启动”和“允许运动”分开管理，避免启动程序即触发非预期动作。

2.5.6.5 协同调度与验证依据

两套机械臂在任务层使用统一的状态管理，在执行层保留不同控制接口。共享作业空间优先采用互斥和顺序执行；发生人工接管、通信故障或状态异常时，停止正常任务推进，并按经过验证的流程进行处理。

系统验证分为三个层次：

计算层：比较同一 ACT 模型在 CPU 与 PS/DPU 混合部署下的端到端延迟、视觉特征误差和动作误差。

控制层：检查指令周期、队列积压、反馈有效性、轨迹执行及超时处理。

任务层：统计抓取、分类放置和结果确认的成功率，并验证双臂作业冲突与异常退出流程。

通过上述划分，系统形成“DPU 承担适合加速的视觉计算、PS 承担策略与任务管理、Spartan-7 承担对应机械臂底层执行”的分层架构。各层通过明确的数据语义、时间约束和反馈机制衔接，为后续部署与性能验证提供接口基础，而不将设计目标直接等同于已实现效果。

2.5.7 跨工程接口边界与结论

表 2-16 跨工程接口边界与结论

总结：

容易混淆的事项	应采用的准确表述
OFDM 包级背压与 RF 样本流	DMA/包缓存支持流控，不代表 RF 采样可任意暂停
HLS 与 DPU 衔接	当前依靠 DDR 缓冲和软件调度，不是全程流接口直连
视觉与导航消息系统	当前视觉采用 ROS2，导航采用 ROS1；不能视为天然互通
检测结果参与导航	本次证据不足以证明检测结果已接入代价地图或运动决策
深度结果	当前为未标定、未配准的近似深度，不能直接当作精确导航障碍距离
软件安全措施	零速度、超时和状态门控不替代物理急停及实车制动验收

本项目按照时序确定性、计算规律性、数据搬运成本和控制复杂度进行软硬

件划分：OFDM 将连续基带处理部署于 PL，将可靠传输和业务协议部署于 PS；视觉将像素处理、神经网络推理及后处理分配给 HLS/DPU，由 CPU 通过共享 DDR 缓冲组织流水线；导航保留软件定位、规划和状态管理，通过受控速度接口连接底盘。接口设计不仅规定数据格式，还明确缓冲所有权、完成条件、异常状态和恢复策略，以避免将局部计算完成误认为端到端业务成功。

2.6 优化过程与性能、资源对比

2.6.1 审计范围与结论

表 2-17 审计范围与结论

三个工程的共同特点不是“模型一次生成正确代码”，而是：

工程	本次识别出的主要优化方向	已有较强证据	仍不能据此证明
OFDM 回环	接收精度、TCP/lwIP 传输、RS 纠错、块交织、视频负载控制	RTL 回归、综合/实现报告、图像和视频板测逐次记录	原始 RF BER、全部信道条件下的收益、长期稳定性
DPU 视觉平台	HLS 精确算术、乘法映射、流水与位宽、平台集成、帧槽和 runner 配置	HLS/C/RTL 测试、布线报告、工件哈希、真实相机运行日志	相对纯 CPU 前后处理的加速比、mAP 提升、通用 YOLO 兼容性
X3plus/KV260 导航	定位门控、反向初始化拒绝、起点书签、故障恢复、任务与速度隔离	单元测试、真实 AMCL/move_base 的隔离运动学集成、历史板端文件核验	实车到点精度、物理刹停距离、现场误拒绝率与所有卡住问题已解决

需求明确化 → 建立观测 → 修改实现 → 增强测试 → 暴露反例 → 修正设计和结论 → 保留版本与证据。

2.6.2 OFDM 工程

2.6.2.1 接收链：从低幅度问题到边界明确的修复

已有记录显示，接收侧改动包括：

保留 FFT、均衡和导频旋转中的有效精度。

在解交织过程中保留 64-QAM 软判决置信度。

针对生成的 Viterbi IP，加入 84 个高置信度零比特前缀，并丢弃对应的 84

个译码输出。

增加可旁路的五点二项式信道平滑。

将 ADC 模型的饱和行为与溢出回绕行为分开测试。

关键结果不是“平滑一定更好”，而是发现了条件性收益。

表 2-18 信道平滑开关对照

仿真条件	平滑关闭	平滑开启	能得出的结论
平坦信道、截断量化、gain=0.02	失败	可通过相应低增益回归	平滑有助于该确定性低幅度场景
平坦信道、四舍五入量化 gain=0.02	通过	通过	量化方式本身影响结果
gain=1、echo_gain=0.25、延迟 4 样本	通过	通过	该短回波场景未暴露劣化
相同增益、回波延迟 16 样本	通过	第 50 字节失败	平滑可能损伤频率选择性较强的信道

边界： 这些是确定性 RTL 仿真，不是硬件灵敏度、BER 曲线或普遍多径性能结论。

2.6.2.2 TCP 图像传输：不能只归因于 Nagle

历史排查从“首包约 0.21 秒延迟、部分图像超时”开始，经历了多轮试验：

仅切换 TCP_NODELAY。

修改传输推进逻辑。

将 TCP_MSS 调整到 1440。

将 TCP_WND 调整到 32768。

进一步检查板端网络定时器和重复报文路径。

中间结果证明：只打开 NODELAY，并不能解决问题。

表 2-19 TCP 图像传输各阶段对照

阶段/配置	成功数	平均成功传输耗时	应用有效吞吐

旧版，Nagle	20/20	426.21 ms	173.36 kB/s
旧版，NODELAY	17/20	404.30 ms	12.97 kB/s
中间修改版，NODELAY	16/20	367.28 ms	9.40 kB/s
MSS=1440 中间版，NODELAY	20/20	379.97 ms	194.46 kB/s
最终 TCP 修复版，Nagle	20/20	159.31 ms	463.45 kB/s
最终 TCP 修复版，NODELAY	20/20	174.45 ms	422.52 kB/s
最终版，NODELAY 连续测试	200/200	160.57 ms	459.80 kB/s

说明：

kB/s 按十进制计算。

有效吞吐包含失败尝试及等待时间，因此不能只看“成功样本平均延迟”。

这些是版本演进记录，不是所有条件均保持不变的单变量实验。

旧版与最终版的 Nagle 两组记录相比，平均延迟下降约 62.6%，有效吞吐为原来的 2.67 倍。这应称为“阶段性观测改善”，不能把全部收益归因于某一行代码。

原始协作记录还明确区分了两个效果：

MSS=1440 后，短测可靠性改善，但首个返回数据仍约延迟 211～214 ms。

接收窗口增至 32768 后，首数据中位延迟降至约 2.1 ms。

本次独立重算了最终 200 次的逐次记录，确认成功数与零交付比特差异和汇总一致。

2.6.2.3 失败优化：把 ACK 重试等待从 100 ms 改为 1 秒

这是很适合写入报告的负向实验：用户提出“是不是等待太短”，随后开展两组各 400 次图像传输。

表 2-20 ACK 等待时间对照实验

指标	ACK 等待 100 ms	ACK 等待 1000 ms

1 秒组只有 399 次成功完成时的计数快照，失败的最后一次没有完整计数，不能当作 400 次的全部计数。

尝试次数	400	400
完整成功图像	400	399
主机截止时间失败	0	1
总耗时	75.009 s	505.319 s
成功图像平均耗时	187.27 ms	1191.02 ms
P95 耗时	274.31 ms	2080.33 ms
应用有效吞吐	394,320 B/s	58,386 B/s
报告的重试数	1,021	1,398*
报告的 RX 丢弃数	975	1,376*

结论：

延长等待没有消除本轮接收失败。

本轮中主要增加了丢失后的恢复等待，吞吐明显下降。

两组顺序运行，RF 环境可能变化，不能把丢弃数量变化全部归因于等待参数。

“重试多”不等于测到了高原始 BER；数据丢失、ACK 丢失和接收提交失败都可能触发重试。

2.6.2.4 RS 与块交织：增加纠错能力，而非无代价提速

优化路径是：

非交织 RS → 平衡缩短码字分配 → 字节交织/反交织 → 独立参考验证 → 真实 FFT/Viterbi 链路验证 → 板端验证。

交织方案将连续字节损伤分散到多个 RS 码字。验证覆盖 1～5 条码字通道及相应纠错边界，而不是只做编码器到解码器的自环。

独立模块实现对比，均为 80 MHz OOC 结果：

表 2-21 RS 编解码模块资源对照

模块/指标	原 RS	加交织/反交织后

编码器 LUT	305	504
编码器 FF	218	235
编码器 RAMB18	2	2
编码器 WNS	6.400 ns	4.551 ns
解码器 LUT	1,110	1,294
解码器 FF	925	969
解码器 RAMB18	1	1
解码器 WNS	4.219 ns	3.515 ns

两模块合计增加 383 LUT、61 FF，BRAM 数量不变；时序余量下降但仍为正。

这属于用一定控制逻辑和时序余量换取突发错误分散能力，不是资源压缩。

验证记录包括：

1,024 种用户长度。

551,888 个参考模型突发窗口。

6,693 个 RTL 包测试。

204 个旧格式回归和 22 个兼容性测试。

14 个 PHY 场景，展开为 21 个速率/场景组合。

测试集内最大译码耗时 12,702 周期，即 80 MHz 下 158.775 μs，不含上游 PHY/Viterbi 延迟。

特别重要的是：原始 PHY 矩阵中保留了旧模式参考退化案例的 passed:false；最终验收将它单列为预期观察，而不是把所有原始测试涂成“通过”。

当前整板报告显示：

表 2-22 交织版整板实现记录

指标	当前交织版整板记录
LUT，布局后报告	37,815

FF，布局后报告	51,157
BRAMtiles	92
DSP	89
布线后 WNS	0.307 ns
布线后 WHS	0.010 ns

不能把这些整板数字与前面的 OOC 模块数字直接混算。

2.6.2.5 板测与视频能力：已验证什么

后续交织版已有真实板测，不能沿用早期文档中的“尚未上板”。

表 2-23 OFDM 板测与视频回环能力

测试	结果	重要限制
UART 长度边界	66/66 通过	指定测试长度和当前配置
UART 连续 1000 字节包	500/500 通过	不暴露完整 RF 重试/丢弃计数
73,944 字节图像，200 次	200/200；589.66 kB/s	本轮报告重试/丢弃为 0
相同大小图像，400 次	400/400；586.72 kB/s	本轮有 5 次重试、5 次 RX 丢弃
640×480、JPEG Q50、目标 7 FPS	120 秒，840/840；实际约 7.00 FPS	有 2 帧迟到，严格实时检查为 false

视频数据来自合成运动图像序列，不是自然摄像头连续采集；节奏测试不包含相机采集、缩放、JPEG 编码和 GUI 显示的完整开销。

因此适合写成：

已验证指定合成 JPEG 负载下，640×480、约 7 FPS 的 120 秒回环传输；840 帧均完整恢复，但有 2 帧超过帧截止时间。

不适合写成：

已保证任意摄像头场景下 640×480、7 FPS 硬实时视频传输。

本次还核对了当前 OFDM 的 bit、XSA、ELF、PS 初始化文件，四项哈希均与交织版板测清单一致。

2.6.3 DPU 视觉平台

2.6.3.1 优化阶段一：精确算术替换

2.6.3.1.1 图像预处理：CDF 归一化

原实现使用较宽通用除法，并用商乘分母恢复余数。优化利用输入帧尺寸上限：

CDF/count 收窄到 22 位。

使用 8 步比较/减法求商。

保留余数，保持 nearest-even 舍入。

不改变接口和均衡语义。

表 2-24 CDF 归一化优化前后对照

指标	优化前	优化后
build_lut 估计延迟	4.002 ns	3.484 ns
build_lut LUT	1,110	708
build_lut FF	889	261
build_lut DSP	3	0
build_lut 最大周期数	12,549	3,333
整 IP LUT	13,543	13,318
整 IP FF	13,299	13,002
整 IP DSP	52	49
整 IP OOC WNS	0.452 ns	0.231 ns

局部 LUT 下降约 36.2%，局部最大周期数下降约 73.4%，但整个 IP 的

WNS 反而下降。

这是一个重要的自纠错案例：

局部算术变好，不代表全局最差路径变好；瓶颈转移到了输入高度校验到状态返回路径。

2.6.3.1.2 YOLO 后处理：NMS 宽乘法

原失败路径涉及 65×17 位 fabric 乘法。单纯增加函数延迟没有真正打断该组合乘法，随后改为：

17 步移位累加，保留完整 82 位乘积。

面积乘法增加明确寄存边界。

保持严格 IoU > threshold 比较。

不引入 top-K 截断或坐标精度削减。

表 2-25 YOLO 后处理 NMS 优化前后对照

指标	优化前	优化后
整 IP LUT	9,260	7,834
整 IP FF	6,068	6,436
DSP	8	8
BRAM tiles	12	12
OOC WNS	0.798 ns	1.535 ns
最差 OOC 数据路径	4.199 ns	3.462 ns
suppress 最大延迟	3 周期	23 周期

LUT 减少约 15.4%，但 NMS 比较延迟明显增加。这里的优化目标是时序/面积权衡，不是无条件提高每帧吞吐。

两核均有算术 PASS 日志和 Verilog 协同仿真 Pass 报告，本次已直接读取。

2.6.3.2 优化阶段二：时序优先的 DSP 分块和位宽约束

用户随后明确允许增加处理周期、优先时序布线。策略因此调整：

图像坐标计算改为有界精确除法。

在完整参数校验之后表达有效位宽，避免提前截断非法输入。

YOLO 面积及阈值乘积采用 DSP 分块、寄存进位级。

不直接迁移到语义未经核验的 Vitis Libraries 算子。

基础平台集成布线对比，不含后续 DPU 链接：

表 2-26 基础平台布线时序与资源对照

指标	2026-10-13 候选	2026-10-14 候选	变化
全局 WNS	0.038 ns	0.157 ns	0.119 ns
图像核 WNS	0.221 ns	0.403 ns	0.182 ns
YOLO 核 WNS	0.038 ns	0.157 ns	0.119 ns
全局 WHS	0.009 ns	0.010 ns	0.001 ns
平台 LUT	27,306	23,104	−15.39%
平台 FF	32,488	31,106	−4.25%
平台 DSP	63	61	−3.17%
BRAM tiles	23.5	23.5	不变

核间资源重分配也很明显：

表 2-27 核间资源重分配

核	LUT：前→后	FF：前→后	DSP：前→后
图像预处理	13,367→12,511	14,353→12,892	55→37
YOLO 后处理	7,808→4,468	7,127→7,205	8→24

这说明“尽量不使用 DSP”不是固定正确策略：YOLO 适当增加 DSP 显著降低 LUT，而图像核又释放了 DSP 预算。

验收边界：

新候选 47,501 条可布线网络全部完成，布线错误为 0。

保留 TIMING-4、TIMING-27 时钟方法学 Critical Warning。

本批初始构建没有完整的构建前哈希清单，后来通过源码、头文件、pragma

与预处理快照匹配补充审计。

旧候选缺少部分新增检查，汇总中的默认 false 不能直接解释为旧候选布线失败。

smoke 协同仿真不是穷尽 RTL 验证。

2.6.3.3 最终 DPU 链接：与基础平台必须分开

最终报告中确实实例化了四类计算模块：

表 2-28 最终 DPU 链接各模块资源

模块	LUT	REG	DSP	URAM
DPUCZDX8G	45,041	80,467	566	40
image_preprocess	12,416	12,827	37	0
sfm_xrt_top	9,616	8,146	14	0
yolo_postprocess	4,371	7,204	25	0
平台部分	9,527	19,507	0	0

最终链接时序报告记录于 2026 年 10 月 14 日：

WNS：+0.001 ns

WHS：+0.007 ns

TNS、THS：0

对应检查中未约束内部端点：0

配置文件可见一个 B3136 DPU、URAM 启用；链接配置为 DPU 300/600 MHz。

不能把基础平台的 +0.157 ns 写成最终 DPU 系统的时序余量。 最终余量只有约 1 ps，属于通过现有约束但余量很薄的结果。

另一个审计细节：XCLBIN 信息中的运行时 kernel 列表主要列出 DPU 和 Softmax，两个 HLS 核属于基础平台集成 IP。不能仅因它们未以相同方式出现在 kernel 列表，就认定硬件里没有它们。

2.6.3.4 软件流水线：减少排队比增加 runner 更有效

真实相机测试使用相同模型和两个 HLS 核，对比 runner 与帧槽配置：

表 2-29 runner 与帧槽配置对照

配置	帧数/时间	FPS	P50 延迟	P95 延迟	最终 RSS
1 runner / 3 槽	188 / 30.16 s	6.23	505 ms	538 ms	279.5 MiB
2 runner / 3 槽	197 / 30.31 s	6.50	486 ms	516 ms	280.5 MiB
1 runner / 2 槽	187 / 30.11 s	6.21	349 ms	381 ms	281.9 MiB
默认配置 90 秒复测	568 / 90.11 s	6.30	349 ms	380 ms	282.5 MiB

结论：

双 runner 短测吞吐仅增加约 4.2%。

3 槽改为 2 槽，中位延迟下降约 30.9%，吞吐基本保持。

少一个帧槽减少 1,424,176 字节应用 CMA 分配。

RSS 并没有同步下降，不能把 CMA 缓冲减少写成整个进程内存下降。

各轮后台负载不同，系统 CmaFree 差值不能直接归因于配置。

90 秒记录中：

568 帧完成推理、深度配对与发布。

最近 256 帧平均：前处理 6.31 ms、DPU 157.35 ms、后处理 34.08 ms。

延迟分位数来自有界最近帧窗口，不是全程所有帧。

阶段耗时包含同步、调度和搬运，不是纯硬件周期。

深度为未标定、未配准的近似值。

未建立纯 CPU 前后处理基线，因此没有可信的“HLS 端到端加速倍数”。

2.6.4 X3plus/KV260 导航

2.6.4.1 优化不是单纯提高精度，而是修正状态与安全语义

表 2-30 导航状态与安全语义优化

问题	早期行为	后续实现

启动位置不可信	种子/旧参数可能被误当作真实位姿	启动与定位确认分离
错误朝向仍被放行	只看激光终点匹配可能误判	增加自由射线与地图障碍矛盾检查
一次短暂异常后永久锁定	清除 initial 后无法自动恢复确认	区分短时异常、暂停、恢复与锁定
同地图复用导致重复初始化	复用时仍 reset	stop 旧任务，但保留健康定位
旧目标/速度跨会话污染	恢复或新目标可能混入旧执行	任务 epoch 与执行 epoch 分离
起点难以重复设置	每次人工猜位置/方向	地图指纹绑定的起点书签

2.6.4.2 反向初始化：从“已有测试通过”到发现实质缺陷

增强测试发现：

真值位置约为 (2.0, 1.0)，方向 0.6 rad。

初始方向反转 180° 后，AMCL 位置误差仅约 0.0309 m。

朝向误差却约 3.1404 rad。

激光终点匹配率 0.7667，高于 0.65 门槛。

门控错误输出 ready=true。

后续不是简单调高 0.65，而是增加射线遮挡矛盾检查：

表 2-31 反向初始化射线检查对照

指标	修复前	加射线检查后
同一错误朝向的终点匹配	约 0.7667	约 0.7667

原终点门槛	0.65	0.65
射线矛盾比例	未用于拒绝	约 0.1667
射线矛盾上限	无	0.08
放行结果	错误放行	拒绝

边界： 对称房间可能存在两个同样符合激光证据的方向；新检查不证明定位唯一性。

2.6.4.3 恢复机制与代价

后续宽松版本采用：

软质量异常容忍 0.8 秒。

连续健康 0.5 秒才清除异常计时。

暂停后 15 秒内恢复并连续健康 1 秒，可自动重新规划原目标。

每任务最多自动恢复 5 次。

主动取消、新目标替换、重新初始化及硬故障不恢复旧任务。

传感器真正过期、无效 TF 等不享受同样的软质量容忍。

这些是可用性与风险之间的参数取舍，不是已经测定的最佳现场参数。放宽数据年龄、匹配门槛会增加容忍，也可能延长错误状态被接受的窗口。

2.6.4.4 验收结果与资源缺口

较晚的 20261020-215644 快照记录：

表 2-32 导航单元测试结果

项目	结果
导航状态机、驱动看门狗、启动与同步	42 项通过
PC 控制台与起点管理	41 项通过
几何/质量	12 项通过
AMCL＋生产门控	29 个阶段通过
起点书签协议	8 个阶段通过

完整 AMCL＋move_base 集成	通过
四个隔离 ROS 容器退出码	均为 0

集成结果：

表 2-33 导航集成定位精度

指标	记录值
第一次终点位置误差	0.01697 m
第一次终点朝向误差	0.09974 rad
改变目标/朝向后的误差	0.07429 m / 0.08617 rad
原地转向朝向误差	0.09629 rad
单次初始化、单个公开目标自动恢复	成功
内部执行 ID 数量	2
暂停时零速度	通过

这些是合成扫描、理想里程计和运动学结果，不是实车厘米级精度。

资源方面没有找到可支持严格前后对照的 CPU 占用、峰值 RSS、ARM 执行耗时分布。容器的“1 GiB、2 CPU”等是限制条件，不是实际资源消耗，不能填进资源节省表。

较晚记录还存在板端文件/参数一致性检查，但后续原始用户对话仍反馈开阔场景卡住。故最终结论应为：

软件状态机和隔离集成验证已明显完善；实车卡住、定位漂移、真实障碍/自遮挡、负载及制动能力仍需现场证据闭环。

2.7 大模型协作记录与自我纠错轨迹

2.7.1 案例一：TCP 问题从单一猜测转向分层定位

用户目标： 提高图像传输可靠性、减少等待。

初始观察：

200 次图像传输中有 14 次超时。

成功样本常有约 0.21 秒首数据延迟。

仅看返回字节没有发现错误，容易误判“链路正常，只是慢”。

协作演进：

模型提出 Nagle/延迟 ACK 等候选解释。

用户要求实际测试，而不是停留在推测。

NODELAY 试验仍出现失败。

MSS、接收窗口分别调整并复测。

首数据延迟明显下降后，继续追查残留 TCP 重传。

后续回答根据 RAM 抓包，指出重复报文路径，而不是继续归咎于 lwIP 核心状态机。

C1 公开回答 msg_w0biRFr0 明确承认阶段边界：

“One issue remains unresolved…”

随后 msg_DpxttXXq 报告了更具体的重复报文证据，并把网络路径与裸机定时器遗漏分开描述。

自我纠错价值：

不把假设写成根因。

不因为一组吞吐提高就终止排查。

将“成功交付”“TCP 重传”“RF 重试”分层统计。

同时保存失败中间版本，而不是只保留最终 20/20。

2.7.2 案例二：ACK 等待不是越长越好

C3 用户原文：

“现在 1s 和 100ms 都做 400 次重传然后对比一下”

消息 ID：7b7e3aba-e86e-41ab-8888-aab23cc2feab

模型首先将含糊的“400 次重传”转成可执行定义：

“1 秒和 100 ms 两个版本各做 400 次连续图片传输，比较实际重传次数。”

随后公开进度记录持续保留反证：

100 ms 组 400/400 成功。

1 秒组前 35 次已发生 55 次丢弃与 55 次重传。

1 秒组 137 次时，已有 340 次重传。

最终 1 秒组 399/400，吞吐明显下降。

纠错轨迹：

等待太短的假设 → 固定图像与硬件对比 → 更长等待没有消除丢失 → 修正结论为恢复等待变长，而非链路质量得到改善。

2.7.3 案例三：HLS 从“省 DSP”转向“时序优先”

C2 用户原文：

“那么请你优化 yolo_postprocess 和 image_preprocess 的关键路径”

消息 ID：9a5f1f04-33a0-4c4a-8869-8f4e55883c00

模型公开回答 msg_rjaQX-Jn 给出前后资源表，同时明确：

“本次完成的是 IP 优化与验证……完整 DPU 工程尚未重新布局布线……”

这避免了把 OOC 成功写成系统完成。

随后用户进一步约束：

“时钟周期可以增加，现在时序布线优先，包括那些大位宽乘除等……”

消息 ID：66f9f0e5-989d-4ba7-bcc7-e8f0c8907e2e

模型在 msg_dsZ9w3te 中公开调整策略：

“我会调整之前‘尽量用 fabric 乘法、给 DPU 留 DSP’的取舍。”

可观察的纠错过程：

初始偏向 fabric 实现，为 DPU 保留 DSP。

发现宽乘法与组合链影响布线时序。

用户明确时序优先、允许额外周期。

改用完整精度 DSP 分块和寄存进位。

以基础平台布线报告验证收益。

保留时钟方法学告警和哈希审计缺口。

后续才更新生产平台、完成链接及板端应用验证。

还出现了验证器自身的纠错：

空 fixture 集合可能造成“全部相同”的误判。

嵌套 TripCount 提取需要修正。

验收加入预期文件数量、非空、哈希、布线完成、RTL Pass 等条件。

这表明模型不仅修改设计，还修正了证明设计正确的方法。

2.7.4 案例四：导航从“测试通过”到主动增强反例

C4 用户原文节选：

“请你再模拟仿真一遍导航的正确性……标定住初始位置，以及方向……”

消息 ID：d02bc4de-9405-4cd9-b8f3-f0d11b1887fb

模型公开回答 msg_w1lYsT4V 没有延续“已经正确”的乐观结论，而是报告：

“位置正确，但初始朝向反了 180°……发现误放行，三轮均复现。”

并明确：

“起点书签功能尚未加入上位机，误放行问题也尚未修复。”

后续轨迹：

原测试正常初始化通过。

增加反向朝向用例。

发现匹配分数高但真实方向错误。

保留失败断言及退出码。

增加自由射线矛盾检查，而不是删断言或只抬高阈值。

加入地图指纹书签与恢复保护。

再测试对称地图，承认无法完全消除多解定位。

这个案例最适合说明：测试通过只证明测试覆盖的条件，不证明问题整体解决。

2.7.5 案例五：安全门控过严引发可用性回退

C4 用户原文：

“为什么……每次 2D Nav Goal 之前都得来一次 2D estimate……”

消息 ID：0b6f96aa-3d25-4739-b623-e5efbf1af649

模型在 msg_JTRghpAV 中识别两条路径：

任意质量异常清除 initial，导致永久锁定。

同地图启动复用仍 reset，造成重复初始化。

先修复为短时异常后可恢复确认，再根据后续要求发展为保留公开任务、内部重新规划的机制。

这是典型的第二阶问题：

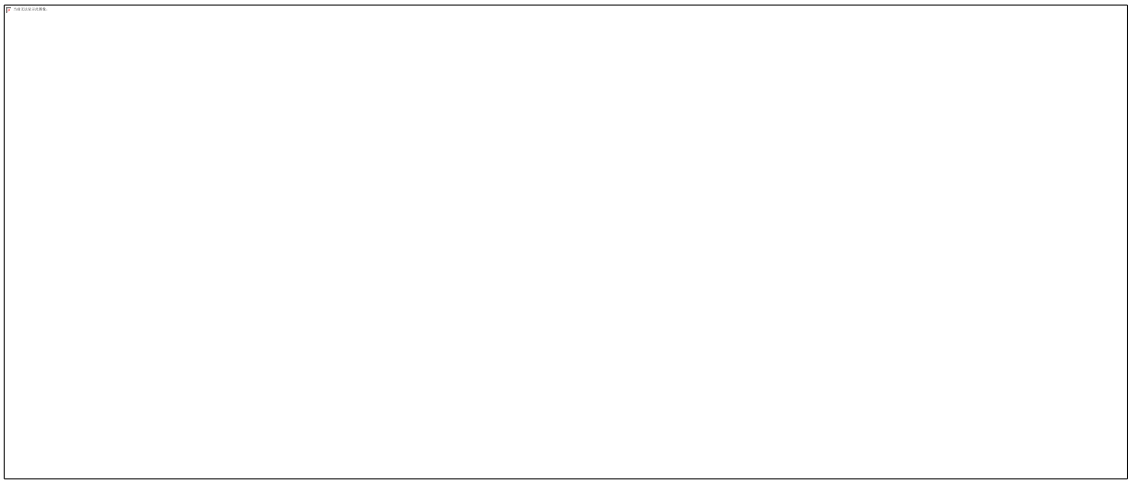
为避免错误定位引入严格门控 → 门控降低危险放行但破坏可用性 → 需

要显式状态机，而不是简单放宽所有检查。

必须保留的负面记录是：后续用户仍反馈“开阔场景卡住”，模型末段也承认板端不可达、仍需核验里程计与融合配置。不能把较早的绿色仿真报告当作后续现场问题已经消失。

2.7.6 实际智能体工作流程

现有记录可概括为以下单智能体闭环：



实际使用的工具职责：

表 2-34 大模型协作工具类别与作用

工具类别	作用
文件读取、搜索	定位接口、限制、历史结果
补丁编辑	修改源码、测试和文档
命令执行	编译、测试、运行 EDA 或采集程序
JSON/日志分析	计算统计、校验计数与失败条件
哈希核验	绑定源码、导出 IP、平台和部署工件

并行启动多个 XSim 或隔离容器属于并行工具任务，不能据此称为多个独立大模型智能体协作。

2.7.7 协作方式的不足

文档阶段混杂。 早期“未部署”和后期“已核验板端文件”同时存在，需要按快照解释。

路径迁移。例如 OFDM 历史记录使用 /home/amd/ofdm/eeSDR_ofdm_loopback，当前目录不同。

会话重复。 恢复会话带入大量历史，统计必须按消息 ID/内容去重。

工具输出缺失。 “命令已启动”或空返回不构成成功证据。

测试语义漂移。 预计失败、参考退化、准备阶段失败必须和产品失败区分。

2.8 技能包的提炼过程、有效性与边界

2.8.1 已存在技能的内容与局限

表 2-35 已有技能内容与局限

技能	已定义的规则	有价值的边界
verilog-optimize	先基线、按关键路径归因、小步修改、standard/aggressive 模式、记录延迟与资源代价	不凭猜测优化；不以牺牲功能或关闭检查伪造收益
vivado-synth-report	确认器件/顶层/约束、打开对应 run、提取时序和资源、失败时保存日志	综合报告不等于布线结果；缺约束时不得认定有效时序

现有技能也有可移植性问题：

部分环境探测示例使用 PowerShell。

工作流依赖特定 MCP 会话接口。

不能假设其命令名称与当前 Linux/Vivado 2022.1 工具封装完全一致。

必须先做工具能力映射，再调用，不能仅因为技能存在就宣称原生兼容。

2.8.2 从失败中提炼技能的证据链

2.8.2.1 技能候选 A：可审计的优化验收

建议名称： evidence-gated-optimization

来源观察：

命令启动不代表完成。

空 fixture 可能造成假通过。

HLS 函数估计改善，不代表整 IP 或整系统时序改善。

历史工件被覆盖后，旧验证清单可能不再匹配当前文件。

提炼规则：

明确基线和候选的器件、时钟、工具、约束及输入。

保存源码/工件身份。

同时检查退出码、成功标记和结构化结果。

预期输出必须存在、非空且数量正确。

核对证据对应的阶段：C 仿真、OOC、基础平台、完整链接、板测。

失败不覆盖旧证据，新的测试使用新目录。

已有有效性证据：

HLS 汇总器增加非空 fixture 及完整文件集合检查。

OFDM 独立验证器核对数量、源哈希、PASS 标记与时序。

本次重新汇总 200、400、840 条记录均与汇总一致。

当前 OFDM 四项工件哈希与板测清单一致。

边界：

哈希一致证明工件身份一致，不证明算法正确；PASS 标记也不能替代测试质量审查。

2.8.2.2 技能候选 B：精确算术的时序/资源优化

建议名称： exact-arithmetic-hls-tuning

来源观察：

通用宽除法成本高。

函数 latency 约束未必拆开宽乘法。

“不用 DSP”可能加重 LUT 与布线压力。

输入校验重构后，综合器可能丢失位宽范围信息。

提炼规则：

先证明输入数值范围。

保留全宽非法参数校验。

在合法域内收窄运算。

明确舍入、饱和、符号和阈值边界。

比较 fabric、串行和 DSP 分块方案。

同时记录 WNS、资源、latency、II 及整帧影响。

已有验证方式：

CDF：2,000 组随机/稀疏直方图及舍入边界。

NMS：20,000 组随机框对及独立宽整数参考。

后续 65×17 乘积：50,000 组检查。

C 仿真、RTL smoke、映射综合、集成布线逐层验证。

ABI、寄存器和导出文件一致性检查。

边界：

精确整数参考一致，不等于与浮点 Python 逐位一致；函数 II=1 也不等于整个应用每周期处理一帧。

2.8.2.3 技能候选 C：通信链路的分层实验设计

建议名称： ofdm-link-diagnosis

来源观察：

零交付字节差异不等于零原始 BER。

增大 ACK 等待反而降低吞吐。

图像文件预分配完整大小不代表所有分片收到。

TCP 重传和无线 ARQ 不是同一个计数。

提炼规则：

区分 PHY、包接收、FCS/RS、ARQ、TCP 和应用层指标。

记录全部尝试，而不是只统计成功样本。

统计失败耗时、缺失分片、计数覆盖范围。

做固定载荷、相同工件的对照。

说明顺序实验与 RF 环境变化的混杂因素。

将实时性、完整性和吞吐分别验收。

已有验证方式：

100 ms/1000 ms 各 400 次对照。

返回数据逐字节、CRC 与 SHA-256 核对。

视频将迟到、跳过和失败分别记录。

840 帧全部恢复，但仍保留 strict_realtime_pass=false。

边界：

应用层完整交付不能替代 SNR/EVM、前纠错 BER 或统计 RF 包错率测试。

2.8.2.4 技能候选 D：独立参考和负向回归

建议名称： independent-oracle-regression

来源观察：

编码器和解码器可能共享同一错误，互相回环仍通过。

仅正常导航无法发现反向初始化问题。

删除失败断言会制造假进展。

提炼规则：

参考实现必须与被测实现尽可能独立。

主动测试越界、复位、背压、乱序、错误初始化。

失败时仍保存结构化结果。

预期参考退化与被测对象失败分开记录。

已发现的反例长期保留。

已有验证方式：

RS 使用独立 Python 多项式除法和映射参考。

RX 直接输入独立生成的损伤字节，避免同源错误互相抵消。

导航加入 180° 反向、对称地图、无效数据和持续故障。

旧模式参考退化仍保留原始失败状态。

边界：

有限随机测试和边界枚举不是形式化等价证明；个别超限 RS 样本被拒绝，不证明所有不可纠正码字都被 CRC 捕获。

2.8.2.5 技能候选 E：机器人状态机和恢复安全

建议名称： navigation-state-recovery

来源观察：

位置误差小不代表朝向正确。

短暂异常清空 initial 造成永久锁定。

旧 action 回调和无时间戳速度可能跨执行污染。

保存地图点不等于知道实车在哪里。

提炼规则：

分离启动、定位确认、任务、内部执行和运动输出。

区分瞬态质量异常与硬故障。

为任务和执行分别设置 epoch。

恢复必须确认旧执行结束和速度静默。

书签绑定地图指纹及人工物理确认。

仿真与实车验收分开。

已有验证方式：

42 项状态机/启动/驱动测试。

41 项控制台/书签测试。

12 项几何检查。

实际 AMCL/move_base 隔离集成。

目标替换、取消、自动恢复和持续故障归零断言。

边界：

ROS1 话题约定不是认证隔离；软件零速度不证明电机实际制动；对称环境不能只靠匹配率消除定位多解。

2.8.2.6 技能候选 F：有界视觉流水线与安全停止

建议名称： bounded-vision-runtime

来源观察：

双 runner 收益有限，增加排队不一定提升吞吐。

减少帧槽明显降低延迟。

候选溢出时截断输出会改变算法语义。

DMA 仍可能写入时释放缓冲会产生风险。

“summary 成功”不代表清理完成。

提炼规则：

使用有界帧槽和最新帧队列。

分别统计吞吐、排队延迟、阶段耗时和内存。

溢出整帧回退，不接受截断检测。

设备完成前不复用或释放缓冲。

停止状态区分正常完成、排空和故障。

验收同时检查 SUMMARY、CLEANUP_OK 和进程退出。

已有验证方式：

runner/帧槽三组对照。

90 秒真实相机运行。

257 候选触发整帧 CPU 回退的硬件自检。

SIGINT 修复后退出码 0、清理标记和无残留检查。

边界：

尚无长期压力、随机 AXI 停顿和完整硬件故障注入验证；深度配对不是标定后的严格 RGB-D 配准。

2.8.3 技能有效性应如何证明

这里要区分两种证据：

2.8.3.1 已有：规则在具体工程中有效

例如：

独立参考发现了同源回环可能漏检的问题。

负向朝向测试暴露真实门控缺陷。

帧槽对照支持低延迟配置选择。

原始逐条记录发现“840 帧成功但 2 帧迟到”的语义区别。

2.8.3.2 缺少：技能包本身提高开发效率的对照

目前不能证明：

使用技能后平均迭代次数下降多少。

首次正确率提高多少。

工具调用或 token 成本下降多少。

在新项目上能否稳定迁移。

是否优于没有技能的同类工作流。

正式验证可采用：

表 2-36 技能包有效性评估维度

评估维度	应记录的指标
正确性	漏检缺陷数、错误通过率、回归失败数
效率	达到验收所需时间、迭代次数、工具调用数
证据质量	缺失工件、错配哈希、未限定结论数量
可迁移性	不同器件、工具版本、项目结构的通过情况
安全性	越权部署、错误复用旧目标、过早释放 DMA 等事件
维护成本	技能版本升级后的回归工作量

应安排“使用技能/不使用技能”的同类任务对照，或者同一失败任务的受控重放。这些是后续验证方案，本次没有执行。

2.8.4 技能边界怎样确定

表 2-37 技能包边界契约

建议把边界写成明确契约，而不是笼统免责声明：

边界类别	必须写清的内容
输入	工程路径、器件、工具版本、时钟、接口、有效数值范围
不变量	协议、ABI、舍入、阈值、复位与错误处理语义
可改项	是否允许增加 latency、使用 DSP、改变内部架构
输出证据	原始日志、汇总、哈希、前后对照、失败案例
停止条件	基线缺失、关键失败、收益不足、工件不一致
禁止外推	仿真→实车、OOC→系统、交付 BER→原始 BER、短测→长期稳定
人工门槛	板卡部署、RF 连接确认、实车物理位置、急停与运动测试

这比“请谨慎操作”更可执行，也更容易自动验证。

2.8.5 综合结论

这三个工程能够支撑一份有说服力的总结，但最值得展示的不是“优化全部

成功”，而是以下事实：

优化收益有真实量化证据。 包括 TCP 阶段性提速、HLS 资源/时序权衡、视觉帧槽降延迟、RS 交织的纠错覆盖。

失败实验被保留下来。包括 1 秒 ACK 等待变慢、预处理局部优化后全 IP WNS 下降、平滑损伤多径场景、导航反向误放行。

大模型协作体现为证据驱动的迭代。 用户不断明确目标与约束，智能体通过工具执行、测试和修订结论，而不是单次代码生成。

可复用方法已经形成，但正式技能化尚未完成。 有脚本、规则、参考模型和验收器；缺少项目专属技能包、明确的版本演化记录及技能效果对照。

最终必须保留边界。 视觉没有 CPU 端到端加速比，导航没有实车精度承诺，OFDM 零交付差异不等于零 RF 误码，7 FPS 试验也不是严格实时全通过。

第三部分　完成情况及性能参数

阐述最终实现的成果（图文结合，实物照片为主）

3.1 整体介绍



图 3-1-1 整车图片

3.2 工程成果

3.2.1 机械成果

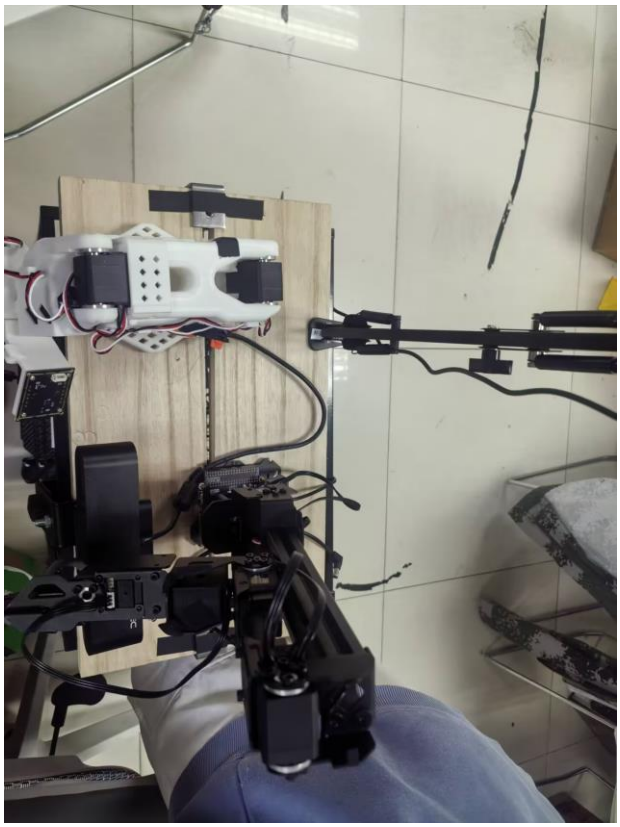


图 3-2-1-1 双机械臂机械图片



图 3-2-1-2 机器人底座机械结构

3.2.2 电路成果

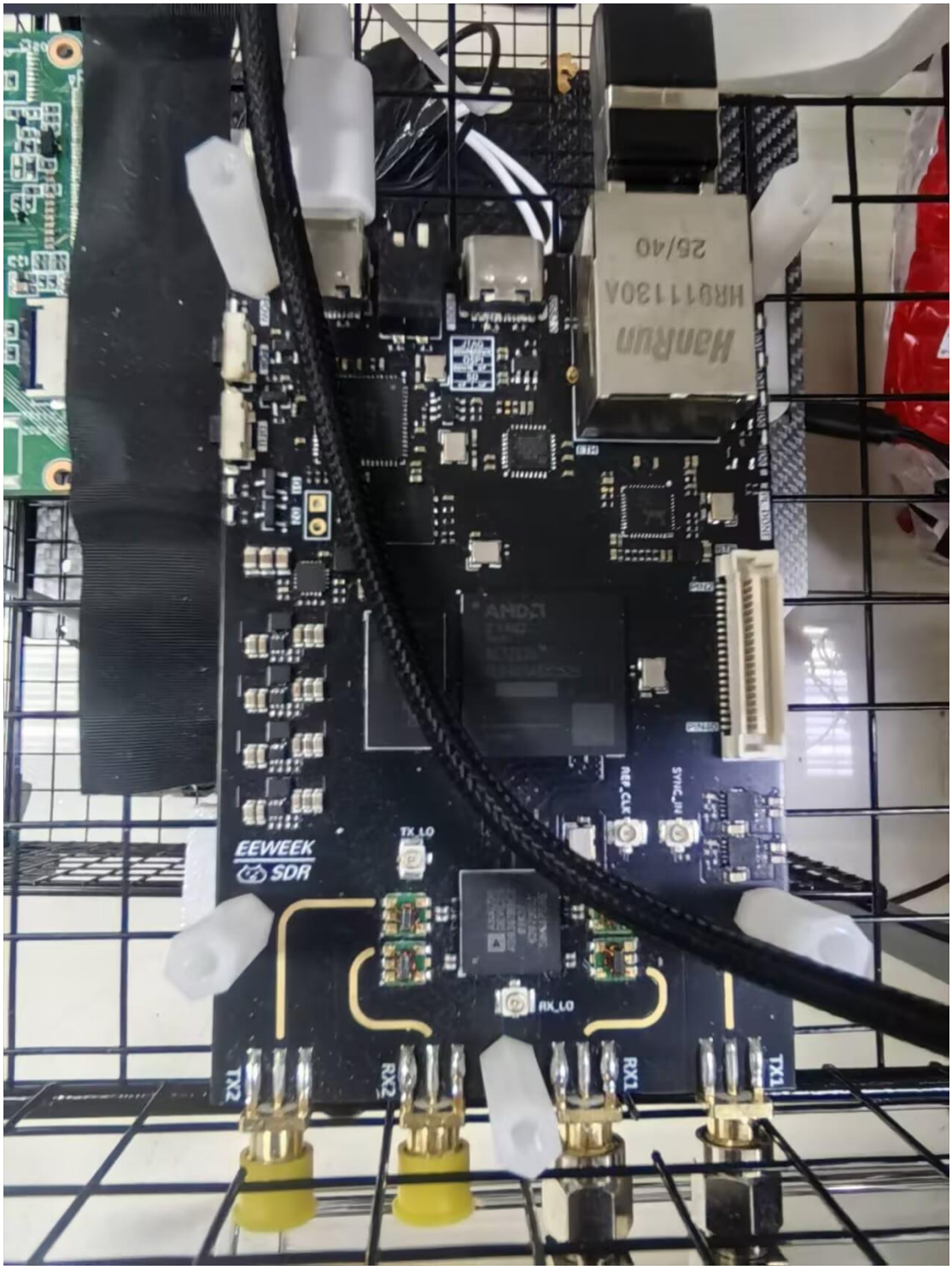


图 3-2-2-1 ZYNQ7020+AD9361 图片

3.2.3 软件成果

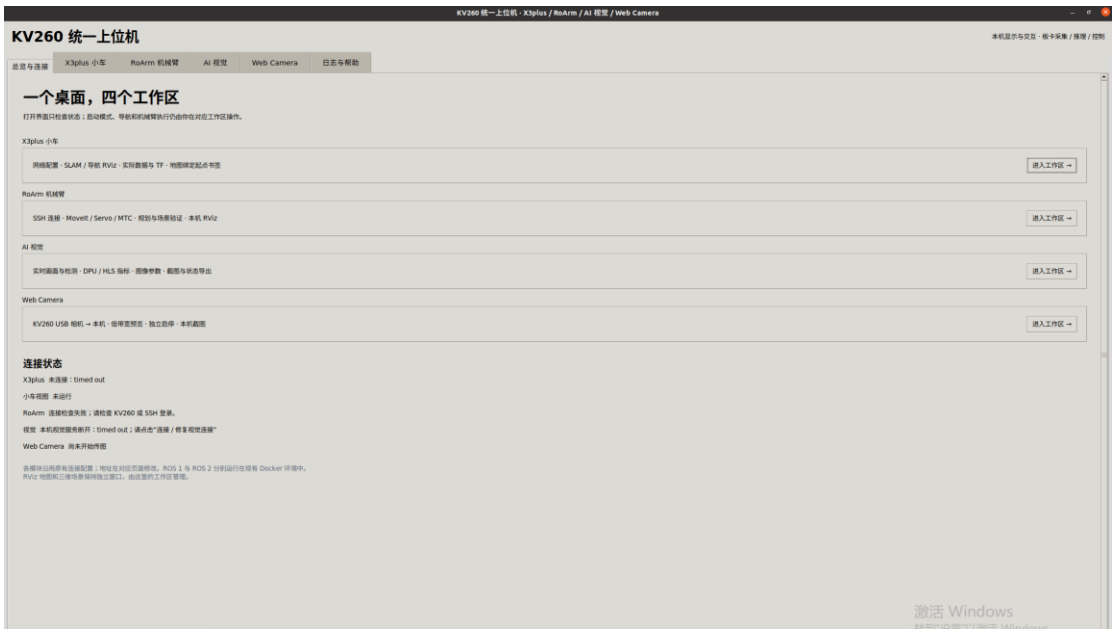


图 3-2-3-1 主机上位机图片



图 3-2-3-2 KV260 上位机图片

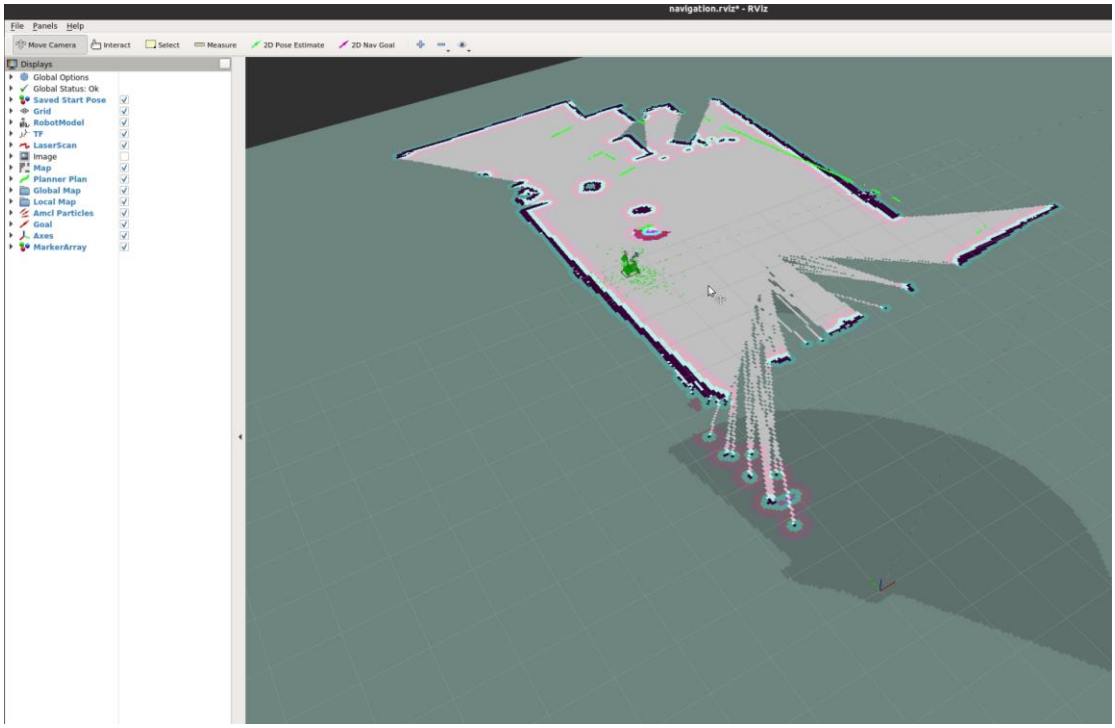


图 3-2-3-3 导航 RVIZ 图片

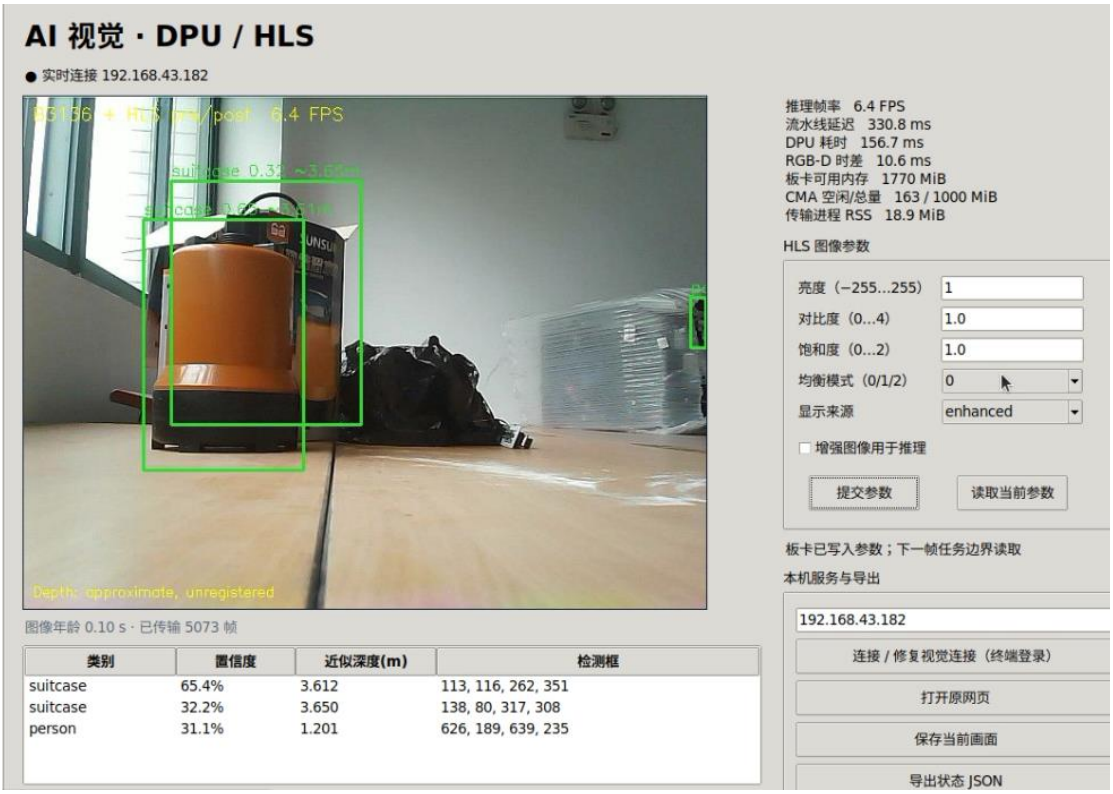


图 3-2-3-4 AI 视觉图片



图 3-2-3-5 OFDM 图传图片

第四部分　总结

4.1 可扩展之处

本作品采用模块化 ROS 架构，可从传感器、执行器、识别类别、导航地图和任务流程五个方向扩展。感知端可增加侧后方激光雷达、力/触觉传感器、条码或 RFID 读取模块，弥补当前约 170 °扫描扇区覆盖和单一视觉感知的不足；执行端可更换夹爪、吸盘或专用工装，适应不同形态物品；识别端可扩展目标类别与多模型调度，但需重新核查 DPU 架构、模型指纹和后处理 ABI。应用场景可由

实验器材整理拓展至小型仓储分拣、生产线物料转运及人员不宜频繁进入的特殊环境辅助作业。通信链路可进一步扩展远程任务下发、状态回传和图像查看功能；FPGA 可扩展更多图像预处理、检测后处理算子。软件方面，可将导航守卫、通信回环验收和证据分级检查固化为可复用的测试清单，便于后续版本迭代与场景迁移。

4.2 心得体会

本作品的研发过程，让我们对“系统集成”有了比单点算法更具体的认识。最初容易把导航、视觉、机械臂和通信分别看成独立模块，但在联调准备中才发现，真正决定系统能否连续作业的，往往是模块之间的接口边界：底盘速度指令是否经过守卫、激光与 TF 是否足够新、视觉缓冲区是否满足对齐与 cache 一致性、候选框溢出后主机能否整帧回退、RS 参数收发两端是否一致、图像分片是否完成 CRC 校验。这些问题不显眼，却直接决定“数据通路是否闭环”。

在移动导航部分，我们逐步理解到配置参数与实测性能不能混写。DWA 的最大速度、目标容差和代价地图分辨率是规划参数，不等于真实定位精度或避障验收结果；inf_is_valid 在特定“有限距离障碍后变为正无穷无回波”场景下将残留致命格由 9 个降为 0 个，也只能说明该场景下旧障碍清除逻辑得到改善，不能扩大为整机避障已验收。因此我们在报告中为每条结论标注“已配置、特定场景验证、离线仿真、板端待实测、不宣称”等证据等级，避免把“支持”写成“已验证”。

视觉链路的制作细节也让我们印象深刻。预处理需要遵守 RGB NHWC INT8 416×416 约定和/256 量化边界，后处理只适用于当前 YOLOv4 416/COCO 风格 ABI；候选超过 256 时硬件不返回截断结果，必须整帧回退软件后处理。两个视觉核经非一致性 HP 端口访问 DDR，主机要检查 64 字节对齐、非重叠、stride 和容量，并在写入后 flush、完成后 invalidate。这些约束促使我们把“能跑通”与“可验收”区分开，先做静态检查和协同仿真，再做板端与回环测试。

通信与机械臂部分同样如此。OFDM 链路不能以 TCP 连接成功或 sendall 成功作为验收依据，而要看 RX 实际 payload 是否逐字节一致、CRC/FCS 是否通过、ACK 窗口是否清空、分片重组是否完整；机械臂也不能仅凭单次检测直接抓取，而要先确认目标、完成坐标转换和抓取条件判断，操作后再复检。双机械臂采用 LeRobot 与 Spartan-7+ROS 的异构控制，让我们体会到平台分工必须服从

任务流程，而不是追求单一平台包办。

在报告整理中，我们反复删除没有证据支撑的“实时”“已验收”“精度达到”等表述，把待实测项如实保留。这个过程虽然繁琐，却让我们形成了更稳妥的工程习惯：先明确边界，再设计实验；先保证接口一致，再谈性能优化；先记录证据，再下结论。后续我们将补齐资源、延迟、功耗、误包率和抓取成功率等板端测试，使系统从“方案闭环”走向“证据闭环”。

第五部分　参考文献

[1] BOCHKOVSKIY A, WANG C Y, LIAO H Y M. YOLOv4: Optimal speed and accuracy of object detection[J/OL]. arXiv preprint, 2020, arXiv:2004.10934. https://doi.org/10.48550/arXiv.2004.10934.
[2] XILINX Inc. DPUCZDX8G for Zynq UltraScale+ MPSoCs: Product guide: PG338 (v3.2)[R]. San Jose: Xilinx, 2021.
[3] AMD/XILINX Inc. Vitis AI user guide: UG1414 (Version 3.5)[R]. San Jose: AMD/Xilinx, 2023.
[4] XILINX Inc. Kria Robotics Stack: A ROS 2-centric approach for hardware acceleration in robotics: WP540 (v1.0)[R]. San Jose: Xilinx, 2021.
[5] CADENE R, ALIBERT S, SOARE A, et al. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch[EB/OL]. (2024). https://github.com/huggingface/lerobot.
[6] QUIGLEY M, CONLEY K, GERKEY B, et al. ROS: An open-source Robot Operating System[C]//ICRA Workshop on Open Source Software. 2009.
[7] MARDER-EPPSTEIN E. move_base ROS package[EB/OL]. http://wiki.ros.org/move_base.
[8] GRISETTI G, STACHNISS C, BURGARD W. Improved techniques for grid mapping with Rao-Blackwellized particle filters[J]. IEEE Transactions on Robotics, 2007, 23(1): 34-46.
[9] GERKEY B P. Adaptive Monte-Carlo localization for ROS[EB/OL]. http://wiki.ros.org/amcl, 2008.
[10] MADGWICK S O H. An efficient orientation filter for inertial and inertial/magnetic sensor arrays[R]. Bristol: University of Bristol, 2010.
[11] THRUN S, BURGARD W, FOX D. Probabilistic robotics[M]. Cambridge, MA: MIT Press, 2005.
[12] VAN NEE R, PRASAD R. OFDM for wireless multimedia communications[M]. Boston: Artech House, 2000.
[13] PRASAD R. OFDM for wireless communications systems[M]. Boston: Artech House, 2004.

[14] REED I S, SOLOMON G. Polynomial codes over certain finite fields[J]. Journal of the Society for Industrial and Applied Mathematics, 1960, 8(2): 300-304.
[15] 王晓君, 刘安. 基于 FPGA 的可调节 OFDM 发射机设计与实现[J]. 现代电子技术, 2025, 48(17): 29-34.
[16] 贺更新, 陈莹. 基于 AD9361 的 MIMO-OFDM 同步定时接收系统设计与性能评估[J]. 吉林大学学报(信息科学版), 2021, 39(6): 630-636.

项目完整复现说明

一、复现范围与验收原则

1.1 复现目标

按依赖顺序完成以下功能：

1. 建立开发主机与 KV260 的软件环境。

2. 运行 OFDM 主机协议测试、DMA RTL/C 回归。
3. 使用配套硬件产物构建 OFDM 固件，完成 UART/TCP 回环。
4. 从源码构建图像预处理和 YOLO 后处理 HLS IP。
5. 部署 KV260 B3136 视觉程序，验证 HLS、DPU、RGB-D 和 ROS2 发布。
6. 验证图像增强、界面控制及视觉图像经 OFDM 回传。
7. 构建 X3plus ROS1 容器，验证驱动、建图、定位和导航。
8. 安装 LeRobot，验证 ACT CPU 推理及模型跨架构迁移。
9. 验证 SO101 网络遥操作协议，再开展经过标定的实机遥操作。
10. 按单独验收流程推进 ACT-DPU 和 Spartan-7 双臂方案。

第 10 项属于待实现或待集成内容，不得与前 9 项已有代码混为一谈。

1.2 结果分类

所有测试结果均使用以下状态之一：

状态	含义
本次软件测试	本次在开发机实际运行的测试，不涉及实体机器人
历史记录	工程中保存的既有日志、报告或测试结果
实机复测	复现者在自己的设备上重新执行并保存证据
设计方案	尚未具备完整可运行实现
模拟占位 / 未实测	仅用于表格或方案说明，不能作为验收结果

1.3 本次实际检查结果

检查对象	本次结果	边界
OFDM 主机测试	54 项中 50 项通过，4 项 GUI 测试跳过	无桌面显示；没有进行 RF 测试
DMA RTL/C 回归	包接口、控制、链路、服务、PHY 适配 5 组通过	编译产物使用临时目录，结束后删除
OFDM XSA 检查	通过	检查 DMA 配置，不代表时序或 RF 性能通过

OFDM 产物一致性	XSA 内 bitstream 与现有两个 TOP.bit 一致，软件平台 XSA 与硬件 XSA 一致	不证明现有源码一定生成这些产物
视觉控制参数测试	2 项通过	不加载 FPGA
视觉工件校验	7 项 SHA-256 一致	不包含全部运行源码
导航单元测试	96 项通过	使用已有 Python 3.10、NumPy 环境；不涉及实车
SO101 网络协议测试	11 项通过	模拟通信与驱动，不移动机械臂

二、硬件、软件与工程准备

2.1 硬件清单

模块	所需硬件
开发环境	Linux x86_64 开发主机、足够存储空间
OFDM	对应 Zynq-7000/AD936x 板卡、供电、JTAG、UART、以太网、符合板卡要求的 RF 连接
视觉	KV260、匹配的启动介质与系统、Astra RGB-D 相机
导航	X3plus 底盘及控制板、适配雷达、供电、可立即使用的急停措施

LeRobot 遥操作	与代码相符的 SO101 主臂、从臂及 USB 串口
Spartan-7 控制	实际控制板、原理图、引脚约束、固件及通信协议；当前需要另行补齐

RF 测试不能直接照搬历史频率或连接方式。首次执行前，应由硬件负责人确认连接、衰减、允许输入功率和实验场地要求。本说明不提供未经核实的 RF 功率设置。

2.2 环境隔离

本项目使用三个相互独立的软件环境：

环境	用途	约束
Vivado/Vitis/Vitis HLS 2022.1	FPGA、HLS 与裸机固件	不自动替换成新版工具
KV260 视觉环境	PYNQ、VART/XRT、ROS2 Humble、Astra	必须与现有 overlay 及模型匹配
ROS1 Melodic 容器	X3plus 驱动、SLAM、导航	与 ROS2 视觉环境分开
LeRobot 独立 Docker/venv	ACT、训练工具、SO101 遥操作	不安装到系统 Python 或视觉 venv 中

ROS Melodic 已结束生命周期；复现时应保留经过验证的镜像和依赖记录，不将联网重新构建成功视为必然。官方最终同步记录位于 2023 年 6 月。

2.3 工程目录约定

以下为**相对于工程根目录**的目录约定（本文统一用 $PROJECT_ROOT 表示工程根目录，复现者自行设置）：

$PROJECT_ROOT/eeSDR_ofdm_loopback
$PROJECT_ROOT/Workspace/HLS_IP/image_preprocess
$PROJECT_ROOT/Workspace/HLS_IP/yolo_postprocess
$PROJECT_ROOT/Workspace/dpu_platform2
$PROJECT_ROOT/kv260_B3136

$PROJECT_ROOT/kv260-launcher-ai
$PROJECT_ROOT/x3plus-kv260-docker $PROJECT_ROOT/lerobot-deploy
板端运行路径按现有脚本约定，相对于板端工程根目录 $BOARD_ROOT：
$BOARD_ROOT/kv260_B3136
$BOARD_ROOT/x3plus-kv260-docker
$BOARD_ROOT/lerobot-deploy $BOARD_ROOT/kv260_astra_ws
这些是**工程路径约定**，不是对复现者用户名或安装位置的硬性要求。更换目录时，必须同时处理脚本、服务文件和构建配置中的路径引用。

2.4 交付包必须包含的内容

仅交付报告或若干 Python 文件不够。至少需要：

1. 源码、测试代码、启动脚本和配置文件。
2. 固定版本信息、依赖清单。
3. 配套模型权重、模型配置和必要的归一化统计。
4. 配套 bitstream、HWH、XCLBIN、XSA 及哈希清单。
5. 相机驱动源码或可恢复的安装环境。
6. 地图文件及其图像文件。
7. 机械臂型号、关节映射、标定流程；标定文件必须属于实际设备。
8. 原始测试日志和性能统计口径。

不要把 SSH 私钥、API 密钥、遥操作共享密钥或网页访问令牌放进公开交付包。

三、基础环境准备

3.1 开发主机基础工具

在适合安装所选 2022.1 工具链的 Linux 开发环境中准备：
sudo apt-get update sudo apt-get install -y
\ git rsync build-essential iverilog
\ python3 python3-venv python3-pip
\ python3-serial python3-pil python3-tk
\ unzip Docker
需单独安装并验证：
docker version docker info

3.2 FPGA 工具链检查

按实际安装位置设置工具根目录环境变量（例如 $XILINX_TOOLS），再执行：
source "$XILINX_TOOLS/Vivado/2022.1/settings64.sh"

source "$XILINX_TOOLS/Vitis/2022.1/settings64.sh"
"$XILINX_TOOLS/Vivado/2022.1/bin/vivado"
-version test -x "$XILINX_TOOLS/Vitis/2022.1/bin/xsct"
test -x "$XILINX_TOOLS/Vitis_HLS/2022.1/bin/vitis_hls"
若安装目录不同，修改本文命令中的工具根目录变量，不修改工程功能逻辑来绕过工具缺失。

3.3 地址与设备配置

现有记录出现过多个网段。本文中的以下地址只是对应历史部署的示例：
开发机执行；必须改成此次实验实际可达的 KV260 地址 export KV260_HOST=ubuntu@<KV260_IP> # 与后文 board_id=10、subnet=137 的 OFDM 固件配置对应 export OFDM_IP=192.168.137.10 先检查连接：
ssh "$KV260_HOST" 'uname -m; cat /etc/os-release; python3 --version'
串口必须人工辨认：
ls -l /dev/serial/by-id/ 不要直接假定：

1. ttyUSB0 一定是底盘。
2. ttyUSB1 一定是雷达。
3. 两个不同启动脚本可以同时打开同一个机械臂串口。

四、OFDM 通信模块复现

依据（均相对于 $PROJECT_ROOT/eeSDR_ofdm_loopback）：

1. host/PROTOCOL.md
2. sim/run_dma_regression.sh
3. firmware/build_dma_sw.tcl
4. firmware/program_dma.tcl

4.1 代码功能对应关系

功能	入口（相对 eeSDR_ofdm_loopback）
DMA UART 数据回环	host/dma_uart.py
TCP 图像回环	host/dma_image.py
TCP 图像连续测试	host/dma_image_benchmark.py
UART 连续测试	host/dma_uart_benchmark.py
JPEG 序列传输测试	host/dma_jpeg_benchmark.py
DMA 图形界面	host/dma_gui.py

历史 mailbox 协议	host/serial_packet.py、host/serial_gui.py

DMA 协议与历史 mailbox 协议不能混用。

4.2 先运行无硬件测试

python3 -B -m unittest discover \ -s "$PROJECT_ROOT/eeSDR_ofdm_loopback/host" \ -p 'test_*.py' -v bash "$PROJECT_ROOT/eeSDR_ofdm_loopback/sim/run_dma_regression.sh" DMA
回归应包含以下通过标志：
DMA_PACKETS_PASS DMA_CONTROL_PASS DMA_LINK_PASS DMA_SERVICES_PASS BOARD_PHY_PASS DMA_REGRESSION_PASS 其中服务测试还可能输出：
DMA_SINGLE_BOARD_SERVICES_PASS 没有桌面显示时，GUI 测试可能跳过。需要验证 GUI 时，应在具备 Tk 和有效桌面显示的会话中重新执行。

4.3 硬件构建边界

当前副本存在配套的（相对 eeSDR_ofdm_loopback）：
build_hw/TOP.xsa build_hw/TOP.bit 但本次未找到历史说明引用的：
build_dma.tcl connect_dma_bd.tcl 因此必须区分：

1. **使用交付产物部署：** 可以继续执行后续步骤。
2. **从 RTL 重新生成全部硬件：** 先补齐经过验证的硬件构建入口、IP 生成配置、约束及导出流程。

部分完整 PHY 仿真脚本还引用了已不存在的旧根目录（旧版绝对路径）。不要将这些脚本直接列为“当前可一键执行”。应先完成路径迁移和生成 IP 依赖检查，再开展全链路 PHY 回归。

4.4 检查 XSA 并重建固件

python3 -B \ "$PROJECT_ROOT/eeSDR_ofdm_loopback/firmware/check_dma_xsa.py" \ "$PROJECT_ROOT/eeSDR_ofdm_loopback/build_hw/TOP.xsa" 预期：
DMA_XSA_PASS: SG TX/RX enabled; external status/control stream disabled 使用一个**尚不存在**的软件工作目录：
export OFDM_SW="$PROJECT_ROOT/eeSDR_ofdm_loopback/build_sw_reproduction" test ! -e "$OFDM_SW" || { echo "目标工作目录已存在，请选择新目录" exit 1 } "$XILINX_TOOLS/Vitis/2022.1/bin/xsct" \ "$PROJECT_ROOT/eeSDR_ofdm_loopback/firmware/build_dma_sw.tcl" \ "$PROJECT_ROOT/eeSDR_ofdm_loopback/build_hw/TOP.xsa" \ "$OFDM_SW" 10 137
这里的 10 137 对应板卡地址配置 192.168.137.10。更换网段时必须同步修改实验网络配置。
固件构建成功应输出：
DMA_SOFTWARE_COMPLETE 4.5 JTAG 加载
此步骤会复位板卡、加载 FPGA 并启动固件。 先停止当前传输，确认只连接目标板卡，并启动对应的 JTAG 硬件服务。
"$XILINX_TOOLS/Vitis/2022.1/bin/xsct"

\ "$PROJECT_ROOT/eeSDR_ofdm_loopback/firmware/program_dma.tcl"
\ "$PROJECT_ROOT/eeSDR_ofdm_loopback/build_hw/TOP.bit"
\ "$OFDM_SW/ofdm_dma/Debug/ofdm_dma.elf"
\ "$OFDM_SW/dma_platform/hw/ps7_init.tcl"
脚本成功标志：
DMA_PROGRAMMED_AND_RUNNING 随后观察串口初始化信息，确认出现 DMA ready。
本文不提供未经核对的 QSPI 永久烧录命令。

4.6 UART 回环

关闭占用该串口的其他工具：
python3 -B \ "$PROJECT_ROOT/eeSDR_ofdm_loopback/host/dma_uart.py"
\ auto --hex 00112233445566778899aabbccddeeff
多个串口无法唯一识别时，使用人工确认的稳定设备路径替换 auto。
验收必须同时满足：
1. 本地入队被接受。
2. 收到对应 RX 返回。
3. 返回数据逐字节一致。
4. 输出 UART LOOPBACK PASS。
“发送成功”“入队成功”不能替代无线回环成功。

4.7 TCP 文件回环

先准备一个实际存在的测试文件：
export OFDM_INPUT="$PROJECT_ROOT/eeSDR_ofdm_loopback/testdata/input.png" export OFDM_OUTPUT="$PROJECT_ROOT/eeSDR_ofdm_loopback/testdata/returned.png" test -s "$OFDM_INPUT" python3 -B
\ "$PROJECT_ROOT/eeSDR_ofdm_loopback/host/dma_image.py"
\ loopback "$OFDM_IP" "$OFDM_INPUT"
\ --output "$OFDM_OUTPUT" sha256sum "$OFDM_INPUT" "$OFDM_OUTPUT" cmp "$OFDM_INPUT" "$OFDM_OUTPUT" 要求源文件和目标文件不同。
成功标准：
1. 输出 LOOPBACK PASS。
2. 板端 CRC 结果有效。
3. 上传 ACK 窗口完成。
4. 两文件 SHA-256 一致。
5. cmp 返回 0。

4.8 性能测试与 GUI

TCP 连续测试：
python3 -B \
"$PROJECT_ROOT/eeSDR_ofdm_loopback/host/dma_image_benchmark.py" \
"$OFDM_IP" "$OFDM_INPUT" \
"$PROJECT_ROOT/eeSDR_ofdm_loopback/reproduction_tcp_results"
\ --count 200 --timeout 30
需要比较 Nagle 设置时，在相同条件下另跑一轮并增加 --nagle，使用不同输出目录。

UART 连续测试：

python3 -B
\ "$PROJECT_ROOT/eeSDR_ofdm_loopback/host/dma_uart_benchmark.py"
\ auto
\ "$PROJECT_ROOT/eeSDR_ofdm_loopback/reproduction_uart_results"
\ --count 2000 --size 1000 --timeout 10

桌面启动 DMA 界面：

python3 \ "$PROJECT_ROOT/eeSDR_ofdm_loopback/host/dma_gui.py" JPEG 序列功能的参数入口：

python3 -B \ "$PROJECT_ROOT/eeSDR_ofdm_loopback/host/dma_jpeg_benchmark.py" \ --help JPEG 帧序列必须来自明确的输入集，记录帧数、帧大小、目标发送速率和过载丢帧；不能将传输一张图片的结果直接称为视频实时传输性能。

五、HLS 前后处理 IP 复现

依据两个工程的 README.md 及实际构建脚本。

5.1 图像预处理

bash
\
"$PROJECT_ROOT/Workspace/HLS_IP/image_preprocess/scripts/check_lut_arithmetic.sh"
HLS_STAGE=all
\ "$XILINX_TOOLS/Vitis_HLS/2022.1/bin/vitis_hls"
\ -f "$PROJECT_ROOT/Workspace/HLS_IP/image_preprocess/scripts/build.tcl" python3
\"$PROJECT_ROOT/Workspace/HLS_IP/image_preprocess/tb/test_host_config.py" python3
\"$PROJECT_ROOT/Workspace/HLS_IP/image_preprocess/tb/test_link_config.py"
HLS_STAGE=all

按脚本执行：

C 仿真 → HLS 综合 → C/RTL 协同仿真 → IP 导出 C 仿真用于执行 C/C++测试平台；协同仿真用于检查生成 RTL 与测试平台的行为，二者不是同一种验证。

主要功能包括：

1. BGR 输入读取。
2. 亮度、对比度、饱和度调节。
3. 当前帧或上一帧亮度均衡。
4. 双线性缩放、Letterbox。
5. BGR 转 RGB、INT8 量化。
6. 原尺寸增强预览。
7. 原图推理与增强预览独立分支。

5.2 YOLO 后处理

在该工程记录的 Linux x86_64 工具环境中：

bash

\ "$PROJECT_ROOT/Workspace/HLS_IP/yolo_postprocess/scripts/check_nms_arithmetic.sh"
LD_PRELOAD=/lib/x86_64-linux-gnu/libm.so.6
\ HLS_STAGE=all
\ "$XILINX_TOOLS/Vitis_HLS/2022.1/bin/vitis_hls"
\ -f "$PROJECT_ROOT/Workspace/HLS_IP/yolo_postprocess/scripts/build.tcl"
LD_PRELOAD 是工程记录中的 x86_64 测试环境处理，不应原样用于 ARM 板端。
主要功能：

1. 三尺度 INT8 输出读取。
2. 置信度过滤、类别选择。
3. Anchor 解码。
4. 可选按类别 NMS。
5. 候选溢出报告。

5.3 输出与接口检查

预期导出目录（相对各自工程根）：
image_preprocess/build/image_preprocess/solution_200mhz/impl
yolo_postprocess/build/yolo_postprocess/solution_200mhz/impl 必须检查：

检查项	要求
目标器件	xck26-sfvc784-2LV-c
HLS 目标周期	5 ns
顶层函数	与对应 IP 一致
AXI-Lite 寄存器	使用本次生成头文件或匹配 HWH
缓冲区	物理地址、容量、对齐、生命周期正确
后处理溢出	整帧回退 CPU，不接受截断结果

PYNQ 缓冲区应使用 device_address 供 PL 访问，并在非一致性路径上正确执行 flush、invalidate 或对应同步操作，不能将普通 NumPy 虚拟地址作为硬件地址。

5.4 集成验收边界

必须分开保存：

1. HLS 综合报告。
2. IP 独立综合报告。

3. 平台布局布线报告。
4. 含 DPU 最终系统的布局布线报告。
5. 对应最终产物的板端测试结果。

HLS 达到 200 MHz 目标，不等于最终 DPU 系统已经在 200 MHz 下完成时序收敛。

现有部分 BD 集成说明描述的是历史 100 MHz 阶段，且含旧路径。不能不核对版本就直接重新执行其中的 apply 或 clock100 操作。

六、KV260 B3136 视觉程序复现

依据（相对 kv260_B3136）：

1. README.md
2. run_vision.sh
3. runtime/hardware.py

6.1 先准备板端基础环境

运行脚本明确依赖以下路径（以板端环境根 $BOARD_ENV 表示）：

$BOARD_ENV/pynq-venv/bin/python3

$BOARD_ENV/ros/humble/setup.bash $BOARD_ROOT/kv260_astra_ws/install/setup.bash 板端先检查：

test -x "$BOARD_ENV/pynq-venv/bin/python3" test -r "$BOARD_ENV/ros/humble/setup.bash" test -r "$BOARD_ROOT/kv260_astra_ws/install/setup.bash" **若任意一项缺失，暂停部署。**

当前 kv260_B3136 目录是应用与工件包，不是完整的空白系统安装器。要让第三方真正从空白 SD 卡开始，需要额外交付：

1. 匹配的 KV260 系统镜像及校验值。
2. PYNQ/PYNQ-DPU、XRT、VART 安装记录。
3. Astra 工作区源码与依赖。
4. 内核、设备树和 CMA 配置记录。
5. 模型来源、量化及编译配置。

不能用“安装最新版 PYNQ”代替这些配套条件。

6.2 复制并校验

开发机执行：

rsync -a

\ "$PROJECT_ROOT/kv260_B3136/"

\ "$KV260_HOST:$BOARD_ROOT/kv260_B3136/"

板端执行：

cd "$BOARD_ROOT/kv260_B3136" sha256sum -c "$BOARD_ROOT/kv260_B3136/artifacts.sha256"

要求 7 项全部通过。

这套程序使用固定的 B3136 模型与 overlay。DPU 模型不能仅按“都是 KV260”判断兼容；编译目标及 fingerprint 必须与硬件对应。

6.3 硬件自检

先停止其他 FPGA/DPU 应用，再执行：

bash "$BOARD_ROOT/kv260_B3136/run_vision.sh" \ --self-test --headless 该步骤会加载 overlay 并执行硬件，不是只读检查。

要求：

1. 出现 SELF_TEST_OK。
2. 无超时或候选处理异常。
3. 正常结束并完成清理。

probe_hardware.py 同样会加载 overlay，不能作为“不会改变硬件状态的探针”使用。

6.4 相机与 ROS2 验证

先确认相机的稳定设备名称，并按实际情况设置 ASTRA_COLOR_DEVICE 或使用 --camera-device。

板端运行：

```
mkdir -p "$BOARD_ROOT/kv260_B3136/logs" bash "$BOARD_ROOT/kv260_B3136/run_vision.sh"
\ --headless
\ --seconds 90
\ --workers 1
\ --slots 2
\ --confidence 0.30
\ --nms 0.60
\ --summary "$BOARD_ROOT/kv260_B3136/logs/reproduction-vision.json"
\ --snapshot "$BOARD_ROOT/kv260_B3136/logs/reproduction-vision.jpg"
```

若已有深度发布节点，增加 --external-depth，避免重复启动。

另一个终端执行：

```
source "$BOARD_ENV/ros/humble/setup.bash" "$BOARD_ENV/pynq-venv/bin/python3" \
"$BOARD_ROOT/kv260_B3136/verify_ros.py"
\ --seconds 45
\ --summary "$BOARD_ROOT/kv260_B3136/logs/reproduction-ros.json"
```

验收内容：

1. RGB 连续采集。
2. 深度消息格式、单位和时间戳有效。
3. 检测序号递增。
4. 坐标、类别与置信度合法。
5. 输出 JSON 报告成功。
6. 出现 CLEANUP_OK，相关进程正常退出。

输出主题：

/yolov4_b3136/detections 该主题是 JSON 字符串消息，不是图像主题。

6.5 性能复现

分别测试：

配置	参数
默认	1. -workers 1 --slots 2
更多帧槽	1. -workers 1 --slots 3
双 runner	1. -workers 2 --slots 3

每轮保持相同模型、阈值、相机配置、测试场景及运行时间，使用不同日志文件。
需要区分：

1. FPS：吞吐率。
2. 帧延迟：一帧从采集后到后处理结束的时间。
3. DPU 阶段时间。
4. HLS 提交、同步、轮询及搬运时间。
5. 有界统计窗口内的分位数。

现有代码和文档说明 RGB-D 深度是**未标定、未配准条件下的近似 ROI 深度**，不能直接作为机械臂抓取坐标。

6.6 安全停止

使用前台 Ctrl+C 或既有安全停止入口。
出现 DRAINING 时：

1. 不直接 SIGKILL 推理主进程。
2. 不并行重载 overlay。
3. 不提前释放仍可能被硬件访问的缓冲区。

以上要求来自本项目的 DMA 缓冲区生命周期设计。

七、视觉界面与 OFDM 图像联动

依据：

$PROJECT_ROOT/kv260-launcher-ai/AI-VISION.md 7.1 当前交付缺口

视觉界面源码在：
$PROJECT_ROOT/kv260-launcher-ai 但预期板端入口在：
$BOARD_ROOT/x3plus-kv260-docker/launcher_vision_ui.py 当前开发机的 X3plus 副本缺少部分启动器文件，包括：
$PROJECT_ROOT/x3plus-kv260-docker/launcher-env.sh
$PROJECT_ROOT/x3plus-kv260-docker/launcher_network.py 部分其他文件可在视觉启动器目录找到，但**不能因此认定任意混合复制后就是完整发布包**。

应先从对应发布版本补齐启动器及固定权限入口；在此之前，使用上一节 CLI 验证视觉程序。

7.2 完整启动器部署后的操作

板端桌面会话执行：

python3 \ "$BOARD_ROOT/x3plus-kv260-docker/launcher_vision_ui.py" 逐项验证：

1. 打开窗口不自动启动相机。
2. 点击“启动视觉”后出现新帧。
3. 调整亮度、对比度、饱和度。
4. 切换当前帧/上一帧均衡。
5. 保持“增强同时用于 DPU 推理”关闭，验证只改变预览。
6. 再启用增强推理，检查消息中的配置编号和参数更新。
7. 保存画面。
8. 安全停止，检查清理结果。

7.3 OFDM 联动

顺序如下：

启动 B3136 视觉 → 确认共享画面持续更新 → 确认 OFDM TCP 客户端空闲 → 采集一张新帧 → 发送 PNG → 接收实际回传文件 → 校验 CRC/SHA-256 → 显示回传文件 必须验证：

1. 视觉停止后不会发送旧画面。
2. 界面显示的是校验通过的返回文件，而非原图。
3. 图像对应的配置、检测与时间戳可追溯。
4. 采集图像和传输图像的耗时分开记录。

这一联动验证的是**视觉图像传输链路**，不代表 OFDM 承担了机械臂实时闭环控制。

八、X3plus 导航模块复现

依据（相对 x3plus-kv260-docker）：

1. README-KV260.md
2. context/src/yahboomcar_nav/launch/yahboomcar_navigation.launch
3. context/src/yahboomcar_nav/scripts/navigation_guard.py

8.1 先运行无硬件测试

使用包含 NumPy 的 Python 环境：

```
python3 -B -m unittest discover
\    -s "$PROJECT_ROOT/x3plus-kv260-docker/navigation-fix"
\    -p 'test_*.py' -v
```

本次系统 Python 缺少 NumPy；换用已有 Python 3.10、NumPy 环境后 96 项通过。

因此复现时要记录**实际解释器路径和 NumPy 版本**，不要只记录命令名 python3。

8.2 补齐启动器依赖

继续前检查：

test -r
"$PROJECT_ROOT/x3plus-kv260-docker/launcher-env.sh" test -r
"$PROJECT_ROOT/x3plus-kv260-docker/launcher_core.py" test -r
"$PROJECT_ROOT/x3plus-kv260-docker/launcher_network.py"
当前副本不能通过上述全部检查，需先补齐同一版本文件。
不要通过创建空文件、注释设备校验或跳过 runtime 检查来强行启动。

8.3 构建 ARM64 容器

补齐发布包后，传到 KV260：
rsync -a
\ "$PROJECT_ROOT/x3plus-kv260-docker/"
\ "$KV260_HOST:$BOARD_ROOT/x3plus-kv260-docker/" 在 KV260 执行：
bash "$BOARD_ROOT/x3plus-kv260-docker/verify-bundle.sh"
bash "$BOARD_ROOT/x3plus-kv260-docker/check-kv260-hardware.sh"
bash "$BOARD_ROOT/x3plus-kv260-docker/build-arm64.sh" 不要把开发机 amd64 镜像或其 catkin 编译产物直接用于 ARM64 板卡。

8.4 配置设备与地图

复制配置：
cp
\ "$BOARD_ROOT/x3plus-kv260-docker/KV260.env.example"
\ "$BOARD_ROOT/x3plus-kv260-docker/KV260.env" 人工配置：

参数	要求
ROBOT_SERIAL	底盘控制板稳定设备路径
LIDAR_SERIAL	雷达稳定设备路径，与底盘不同
LIDAR_BAUDRATE 等	与实际雷达型号一致
ROS_MASTER_URI	远程访问时使用板端实际 IP
ROS_HOSTNAME	对其他 ROS 节点可达

MAP_DIR	与容器 /maps 挂载源一致

使用受控权限建立地图目录，不必按旧文档给整个目录 0777 权限。

8.5 启动前的机械臂风险

本次直接核对到：
$PROJECT_ROOT/x3plus-kv260-docker/context/src/yahboomcar_bringup/scripts/Mcnamu_X3plus.py 其初始化会下发：[90, 145, 0, 45, 90, 30] 并调用舵机角度数组设置。
因此启动底盘驱动并不等于“只读启动”，可能同时引起机械臂动作。
首次启动必须清空工作区、确认初始姿态安全，并按设备操作规程处理动力和急停。本文没有修改该行为。

8.6 启动基础服务

满足安全条件后：
source "$BOARD_ROOT/x3plus-kv260-docker/KV260.env"
bash "$BOARD_ROOT/x3plus-kv260-docker/run-master.sh"
bash "$BOARD_ROOT/x3plus-kv260-docker/run-robot.sh"
bash "$BOARD_ROOT/x3plus-kv260-docker/start-hardware.sh" 检查：
docker exec x3plus-robot bash -c '
source "$BOARD_ENV/ros/melodic/setup.bash"
source "$BOARD_ENV/x3plus_ws/devel/setup.bash" rosnode list rostopic list '
要求存在有效的：
/scan /odom /cmd_vel TF 注册了话题不等于实际收到新消息；还应检查数据时间戳和 TF 连通性。

8.7 建图与地图保存

启动 SLAM：
bash "$BOARD_ROOT/x3plus-kv260-docker/start-slam.sh" 开发机图形桌面启动查看器：
bash "$PROJECT_ROOT/x3plus-kv260-docker/remote-rviz/start-pc-rviz.sh" 在受控低速运动条件下建图，完成后保存：
docker exec x3plus-robot
bash -c ' source "$BOARD_ENV/ros/melodic/setup.bash"
source "$BOARD_ENV/x3plus_ws/devel/setup.bash"
rosrun map_server map_saver -f /maps/reproduction_map ' 同时保存并交付：
地图 YAML 地图 PGM 地图版本/哈希 建图日期 雷达与里程计配置 停止 SLAM 后再启动基于保存地图的导航，不同时保留两个地图定位体系。

8.8 当前导航参数的重要区别

实际源码默认：
enable_multipoint = false quality_interlock = false 因此：
1. 普通导航不会自动启动循环多点巡航。
2. 不能将旧说明中的“所有定位质量异常都会停车”当作当前默认行为。

3. quality_interlock=false 时，确认定位后的部分质量异常仅作为告警处理。

若本次验收要求启用严格质量联锁，应在没有其他导航实例时，明确启动：

```
docker exec -it x3plus-robot bash -c '
source "$BOARD_ENV/ros/melodic/setup.bash"
source "$BOARD_ENV/x3plus_ws/devel/setup.bash"
roslaunch yahboomcar_nav yahboomcar_navigation.launch
\       use_rviz:=false
\       map:=/maps/reproduction_map
\       enable_multipoint:=false
\       quality_interlock:=true '
```

该配置是本次验收的显式选择，不是原默认值，也不构成硬件安全认证。

8.9 导航验收

开发机启动导航查看器：

```
bash
\   "$PROJECT_ROOT/x3plus-kv260-docker/remote-rviz/start-pc-rviz.sh"
\   navigation --replace
```

按顺序操作：

1. 人工确认车辆实际位置和朝向。
2. 用 2D Pose Estimate 初始化。
3. 等待定位状态满足要求。
4. 发送短距离、明确终点朝向的目标。
5. 检查公开任务状态、内部任务状态和实际运动。
6. 测量终点位置与朝向误差。
7. 验证取消、停止、新目标替换和异常后的恢复行为。

查看状态：

```
docker exec x3plus-robot bash -c '  source "$BOARD_ENV/ros/melodic/setup.bash"
rostopic echo -n 1 /navigation/localization_status '
```

需要分别检查：

```
/move_base/status /navigation_internal/move_base/status /cmd
```